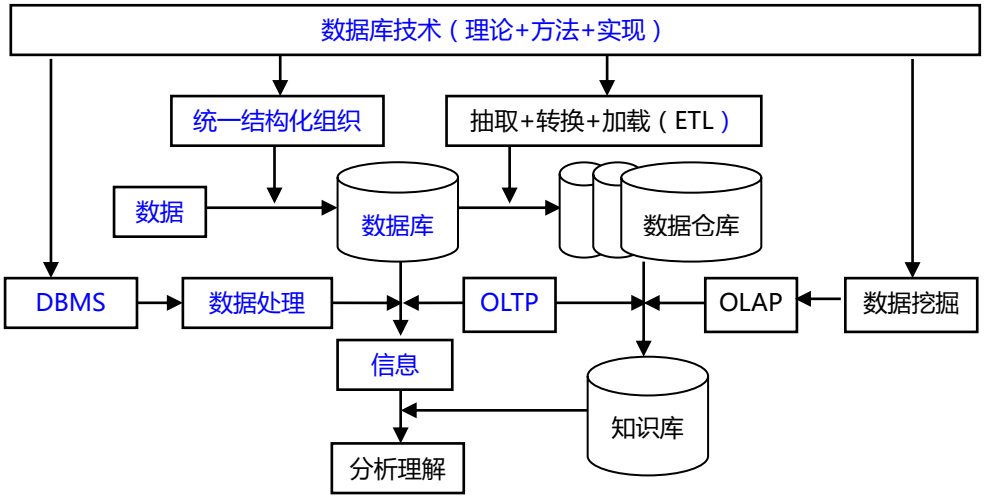
数据库系统原理

习题与实验

HappyYou

韩培友

数据库技术



2016.6.6

第 1 章习题

1.1 名词解释：数据、数据库、数据库管理系统/关系数据库管理系统、数据库系统/关系数据库系统、属性　实体、联系、元组、关系、关系模式、候选键、主键。

数据 Data 是表示客观事物的符号。

数据库 DB 是长期存储在计算机内，有组织可共享的大量数据的集合。即：存放数据的电子仓库。

关系数据库 RDB 是支持关系模型的数据库。即：所有关系的集合。

数据库管理系统 DBMS 是提供给用户，并帮助用户建立、使用和管理数据库的软件系统。

关系数据库管理系统 RDBMS 是支持关系数据库的数据库管理系统。

数据库系统 DBS 是在计算机系统中引入数据库后，由数据库、数据库管理系统、开发工具、应用系统、数据库管理员和用户等构成的完整系统。

关系数据库系统 RDBS 是利用关系数据库管理系统管理关系数据库的数据库系统。

属性（字段，数据项）是实体所具有的特性。

实体（记录，元组）是客观存在且相互区别的事物。

联系是实体集之间或者实体集内部的关联关系。

关系是笛卡尔积的子集。由关系模式和元组集合两部分组成的整体。

关系模式是组成关系的关系型，即：关系的结构描述。

候选键（候选码）是能够区分每一个实体的最小属性集。

主键（主码）是被选中使用的候选键。

1.2 简述数据库管理系统的功能。

（1）数据定义

（2）数据操纵

（3）事务和运行管理

（4）组织、存储和管理数据

（5）数据库的建立和维护

1.3 简述数据库系统的组成和特点。

数据库系统由硬件、软件和人员等三大部分组成。

（1）硬件：计算机硬件和数据库专用硬件。

（2）软件：操作系统 OS、DBMS、开发工具（程序设计语言和专用工具）和应用系统等。

（3）人员：数据库设计员、程序员、数据库管理员和用户等。

数据库系统的特点是数据永久结构化存储、冗余度低，独立性高、共享和易扩展等。

1.4 解释数据模型，常用的数据模型有哪些？简述数据模型的组成要素。

数据模型是实际问题的模拟和抽象。即：针对实际问题，研究数据及其联系，并最终解决

问题的方法和步骤。

常用的数据模型为层次模型、网状模型、关系模型和面向对象模型等。

数据模型由数据结构、数据操作和数据完整性约束等组成。

1.5 简述 DBA 的职责。

（1）决定数据库的信息内容和结构。

（2）决定数据库的存储结构和存取策略。

（3）定义数据的安全性要求和完整性约束条件。

（4）监控数据库的使用和运行。

（5）数据库的改进、重组和重构。

1.6 简述数据管理技术发展的基本阶段。

数据管理技术的发展阶段为人工管理、文件管理和数据库系统等。

1.7 简述文件系统与数据库系统的区别。

文件系统：数据以文件方式存储，使用文件管理系统对数据文件进行统一组织、存储和管理，加快了数据处理速度，提高了准确性。特点如下：

（1）数据可以保存，没有结构化。

（2）数据不能共享。

（3）文件系统管理数据。

（4）数据不具独立性。

（5）数据冗余高。

数据库系统：利用数据库管理系统，把所有的数据进行有组织的统一结构化管理。使数据处理的速度更快，准确性更高，尤其是利用计算机处理大量的数据时，更体现出了数据库系统阶段在数据处理方面的快捷、准确、简单和方便等优势。特点如下：

（1）数据结构化且易扩充。

（2）数据库管理系统管理数据。数据库管理系统对数据进行统一管理和控制；同时提供数据安全性保护、数据完整性约束、多用户并发控制和数据库恢复等完善的数据保护机制。

（3）数据独立性高。

（4）数据共享性高。

（5）数据冗余度低。

1.8 解释概念模型和 E-R 图，简述概念模型的组成要素。

概念模型是数据及其关系的图形表示。即：利用专用描述工具,按照统一的描述方法，对实际问题抽象后，而建立的体现数据及其关系的结构模型。

E-R 图是使用实体-联系方法所建立的用于描述概念模型中实体及其联系的图形。

概念模型的组成要素是实体、属性和联系等。

1.9 解释实体之间的常用联系。

一对一联系(1:1)；一对多联系(1:n)；多对多联系(m:n)。

1.10 简述数据库系统的模式结构。

外模式、模式和内模式三级模式以及外模式/模式和模式/内模式二级映像等组成。

1.11 解释数据独立性。数据独立性包括哪两种独立性？简述数据库系统模式结构与数据独立性的关系。

数据独立性：应用程序和数据之间相互独立，互相不受影响。具体包括数据的物理独立性和数据的逻辑独立性。

（1）数据的物理独立性：如果数据库的内模式需要修改，即数据库的存储结构和存取方法有所变化，那么通过模式/内模式映象的相应修改，使模式尽可能保持不变。亦即对内模式的修改尽量不影响模式、外模式和应用程序。

（2）数据的逻辑独立性：如果数据库的模式需要修改，即增加记录类型或者增加数据项，那么通过外模式/模式映象的相应修改，使外模式尽可能保持不变。亦即对模式的修改尽量不影响外模式和应用程序。数据的物理独立性和逻辑独立性合称为数据独立性。

1.12 简述数据库技术的主要研究领域和应用领域。

研究领域：数据库技术理论研究、数据库设计、DBMS 研发、数据库应用系统研发等。

应用领域：计算机辅助教学；计算机辅助设计；计算机辅助制造；公安；银行；税务；农业；航空航天航海；电子商务（网上商城）；程序外包、手机游戏和网络游戏的外挂程序等嵌入式数据库应用；飞机售票系统、火车售票系统、银行管理系统、财务管理系统和税务管理系统等专用数据库应用；超市营销数据分析系统、证券投资分析系统、股票分析与预测系统和电信服务分析与预测系统等。

第 2 章习题

2.1 简述关系数据语言分类。

关系数据语言分为关系代数语言和关系演算语言；关系演算语言又分为元组关系演算语言和域关系演算语言等。即：

$$\text{关系数据语言}\begin{cases}\text{关系代数语言(例如：ISBL)}\\ \text{关系演算语言}\begin{cases}\text{元组关系演算语言(例如：ALPHA，QUEL)}\\ \text{域关系演算语言(例如：ISBL)}\end{cases}\\ \text{关系代数/演算双重语言：结构查询语言（例如：SQL Server）}\end{cases}$$

2.2 简述关系运算基本运算，导出运算和核心运算。给出使用基本运算表示导出运算的表示方法。

基本运算：笛卡尔积、并集、差集、选择和投影等。

导出运算：交集、连接和除等。

$R \cap S = R - (R - S)$

$R \underset{R.A\theta S.B}{\bowtie} S = \sigma_{R.A\theta S.B}(R \times S)$

$R(X) \div S(Y) = \pi_{X-(X\cap Y)}(R) - \pi_{X-(X\cap Y)}((\pi_{X-(X\cap Y)}(R) \times \pi_{X\cap Y}(S)) - R)$

核心运算：选择、投影和连接等。

2.3 简述等值连接和自然连接的的与区别和联系。

等值连接：连接条件取等值的连接运算。

自然连接：具有公共属性且去掉重复列的等值连接。

等值连接和自然连接都是要求属性间按照等值进行连接，而自然连接要求两个关系中相等的分量必须是相同的属性组（存在公共属性）而等值连接不需要，且自然连接要在结果中把重复的属性列去掉而等值连接不需要。

2.4 计算 R（A，B，C，D，E）÷ S（D，E）=Q（A，B，C）=?

R 的元组如下：

{（y，a，z，b，3），（z，a，z，a，1），（z，a，z，b，1），（z，a，y，b，1）}

S 的元组如下：

{（a，1），（b,1）}

答案：Q（A，B，C）={（x，a，z），（z，a，z）}。

2.5 已知工程管理数据库的关系如下：

供应商 S（供应商号 SNo，供应商名 SName，所在城市 City）

零件 P（零件号 PNo，零件名 PName，颜色 Color，重量 Weight）

工程 J（工程号 JNo，工程名 JName，所在城市 City）

供应 SPJ（SNo，PNo，JNo，供应数量 Qty）

请完成如下查询：

（1）查找上海的各工程名。

（2）查询供应商 S1 给工程 J2 供应的所有零件名。

（3）查询北京的工程使用蓝色零件的工程号。

（4）查询没有使用 S1 所供应零件的工程号。

（5）查询找出能提供零件号为 P3 的供应商。

（6）查询能提供供应商 S1 所提供的所有零件的供应商

（7）查询不提供零件 P1 和 P2 的供应商。

（8）查询同时提供零件 P1 和 P2 的供应商。

（9） 查询同供应商 S1 在同一城市的供应商所提供的所有零件。

（10）查询供应红色的 P1 零件的供应商，且其供应量大于 1000。

答案：

（1）$\pi_{JNAME}(\sigma_{CITY='上海'}(J))$

（2）$\pi_{PNAME}(P \bowtie \sigma_{SNO='S1' \wedge JNO='J1'}(SPJ))$

（3）$\pi_{JNO}(\sigma_{COLOR='蓝色'}(P) \bowtie SPJ \bowtie \sigma_{CITY='北京'}(J))$

（4）$\pi_{JNO}(J) - \pi_{JNO}(\sigma_{SNO='S1'}(SPJ))$

（5）$\pi_{SNo}(\sigma_{PNo='P_3'}(SP))$

（6）$\pi_{SNo,PNo}(SP) \div \pi_{PNo}(\sigma_{SNo='S_1'}(SP))$

（7）$\pi_{SNo}(SP) - (\pi_{SNo}(\sigma_{PNo='P_1'}(SP)) \cup \pi_{SNo}(\sigma_{PNo='P_2'}(SP)))$

（8）$\pi_{SNo}(\sigma_{PNo='P_1'}(SP)) \cap \pi_{SNo}(\sigma_{PNo='P_2'}(SP))$

（9）$\pi_{PNo}((\pi_{SNo,City}(S) \div \pi_{City}(\sigma_{SNo='S_1'}(S))) \rhd\lhd SP)$

（10）$\pi_{SNo}(\sigma_{Color='红色' \wedge Qty>1000}(P \rhd\lhd SP))$

2.6 已知读者订单管理数据库的关系如下：

读者 Read（读者号 RNo，读者名 RName，读者身份 Status，读者城市 City）

图书 Book（书号 BNo，书名 BName，书价 Price，出版社 Publisher）

订单 OrderForm（订单号 ODNo，BNo，订书数量 Quantity）

订单读者索引 Index（ODNo，RNo）

请完成如下查询：

（1）找出科学出版社出版的全部书号和价格。

（2）找出有订单的书名和订单号。

（3）找出订购数量为 20 的所有订单号和书号。

（4）找出至少订购了“数据库技术”的读者号。

（5）找出没有订购任何图书的读者号。

答案：

（1）ISBL：$\pi_{BNAME,PRICE}(\sigma_{PUBLISHER='科学出版社'}(Book))$。

（2）ISBL：$\pi_{ODNO,BNAME}(OrderForm \bowtie Book)$

（3）ISBL：$\pi_{ODNO,BNO}(\sigma_{QUANTITY=20}(OrderForm))$

（4）ISBL：$\pi_{RNO}(\sigma_{BNAME='数据库技术'}(Index \bowtie OrderForm \bowtie Book))$

（5）ISBL：$\pi_{RNO}(Read) - \pi_{RNO}(OrderForm)$

2.7 已知商品销售数据库的关系如下：

商品 SP（商品号 SPNO，商品名 SPN）、库存 KC（SPNO，库存数量 SL1）

销售 XS（SPNO，KHNO，销售日期 RQ，销售数量 SL2，单价 DJ）

客户 KH（客户码 KHNO，客户名 KHN）。

请完成以下查询：

（1）查询所有商品的全部信息。

（2）查询客户“蓝天食品厂”所购的商品码和数量。

（3）查询客户‘1616’所购商品的库存数量。

（4）查询购买所有商品的客户名。

答案

（1）$\pi_{SPNO,SPN}(SP)$

（2）$\pi_{SPNO,SL2}(\sigma_{KHN='蓝天食品厂'}(KH \bowtie XS))$

（3）$\pi_{SL1}(\sigma_{KHNO='1616'}(KC \bowtie XS))$

（4）ISBL：$\pi_{KHNO,SPNO}(KH \bowtie XS) \div \pi_{SPNO}(SP)$

2.8 已知运输公司数据库的关系如下：

Team（TNo，TName，Phone）

Car（CNo，Company，CDate，TNo）

Driver（DNo，DName，Phone，TNo，WDate）

代码含义：

Team-车队：TNo-车队编号；TName-车队名称；Phone-电话

Car-车辆：CNo-车辆编号；Company-生产厂家；CDate-出厂日期

Driver-司机：DNo-司机编号；DName-姓名；Phone-电话；WDate-聘期

请完成如下查询：

（1）查询生产厂家是“金龙公司”的车辆信息，显示数据为 CNo，Company，CDate，TNo。

（2）查询李明司机的名称、电话和聘期，显示数据为 DName，Phone，和 WDate。其中：Phone 为司机的电话。

（3）查询车队编号为“01”的车辆信息，显示数据为 TName，Phone，CNo，Company

和 CDate。其中：Phone 为车队的电话。

（4）查询车队名称为“翔龙车队”的司机信息，显示数据为 TName， Phone（该电话为车队电话，DName 和 Phone（该电话为司机电话）。

（5）查询运输公司车队、车辆和司机的信息清单，显示数据为 TName，Phone（该电话为车队电话，CNo，Company，CDate，DName，Phone（该电话为司机电话）和 WDate。

答案：

（1）$\pi_{\text{CNo，Company，CDate，TNo}}$（$\sigma_{\text{Company=‘金龙公司’}}$（Car））

（2）$\pi_{\text{DName，Phone，WDate}}$（$\sigma_{\text{DName=‘李明’}}$（Driver））

（3）$\pi_{\text{TName，Phone，CNo，Company，CDate}}$（$\sigma_{\text{TNo=‘01’}}$（Team $\bowtie$ Car））

（4）

$\pi_{\text{DName,Team.Phone,DName,Driver.Phone}}(\sigma_{\text{TName=‘翔龙车队’}}(\text{Team} \bowtie \text{Driver}))$

（5）

$\pi_{\text{TName,Team.Phone,CNo,Company,CDate,DName,Drive.Phone,WDate}}(\text{Team} \bowtie \text{Car} \bowtie \text{Driver})$

2.9 已知学籍管理数据库的关系如下：

学生 S（学号 SNo，姓名 SName，性别 SSex，年龄 SAge，班级 Class，学院 Coll）

课程 C（课程号 CNo，课程名 CName，选修课 CPNo，学分 Credit）

选课 SC（学号 SNo，课程号 CNo，成绩 Grade）

请完成如下任务：

（1）查询男生的基本信息。

（2）查询学生的姓名、年龄和班级。

（3）查询物流 1602 班的学生的学号。

（4）查询选修英语的学生的姓名。

（5）查询选修所有课程的学生的学号和姓名

（6）把学号、课程号和成绩分别为 2016010201、020102 和 96 的元组添加到选课 SC。

（7）在选课 SC 中，删除学号为“2016010201”、选修课为“网球”的元组。

（8）查询“信息学院”的学生信息。

（9）查询“外语学院”，并且年龄小于等于 16 岁或者大于 22 岁的学生信息。

（10）查询学生的姓名和所在学院。

（11）查询“信息学院”的学生的学号和姓名。

（12）查询“软件学院”，且年龄小于等于 16 或者大于 22 岁的学生的姓名、性别和年龄。

（13）查询选修了课程的学生的学号。

（14）查询年龄在 19 ~ 22 岁（包括 19 岁和 22 岁）之间的学生的姓名、系别和年龄。

（15）查询外语学院、数学学院和工商学院学生的姓名和性别。

（16）查询至少选修课程号分别为：020101、030102 和 060206 三门课程的学生学号。

（17）查询选修课程号为 020101 或者 030102 或者和 060206 的课程的学生的学号。

（18）查询只选修课程号分别为：020101、030102 和 060206 三门课程的学生的学号。

（19）查询选修了“数据库原理”课程的男生的学号和姓名。

（20）查询至少选修了一门课程其先修课的课程号为 2016010201 的学生的姓名。

答案

（1）$\sigma_{SSex='男'}(S)$

（2）$\pi_{SName,Age,Class}(S)$

（3）$\pi_{SNo}(\sigma_{Class='物流1602'}(S))$

（4）$\pi_{SName}(\pi_{CNo}(\sigma_{CName='英语'}(Course)) \bowtie SC \bowtie Student)$

（5）$\pi_{SNo,SName}(Student \bowtie (\pi_{SNo,CNo}(SC) \div \pi_{CNo}(Course)))$

（6）$SC \cup \{'S01','C02','B'\}$

（7）$SC-((\pi_{CNo}(\sigma_{CName='网络'}(C))) \bowtie \sigma_{SNo='S02'}(SC))$

（8）$\sigma_{Coll='信息学院'}(S)$

（9）$\sigma_{SDept='外语学院' \wedge (SAge \le 16 \vee SAge > 22)}(S)$

（10）$\pi_{SName,Coll}(S)$

（11）$\pi_{SNo,SName}(\sigma_{Coll='计信息学院'}(S))$

（12）$\pi_{SName,SSex,SAge}(\sigma_{Coll='软件学院' \wedge (SAge \le 16 \vee SAge > 22)}(S))$

（13）$\pi_{SNo}(SC)$

（14）$\pi_{SName,SDept,SAge}(\sigma_{SAge \ge 19 \wedge SAge \le 22}(Student))$

（15）$\pi_{SName,SSex}(\sigma_{SDept='外语系' \vee SDept='数学系' \vee SDept='计算机系'}(Student))$

（16）建立包含三个课程号的临时关系 T，$\pi_{SNo,CNo}(StudentCourse) \div T$。

（17）使用并集。

（18）使用差集：至少选三门 − 不选这三门。

（19）$\pi_{SName}(\sigma_{CName='IDL' \wedge SSex='男'}(Student \bowtie StudentCourse \bowtie Course))$

（20）$\pi_{SName}(\sigma_{CPNo='1990010101'}(Student \bowtie StudentCourse \bowtie Course))$

2.10 解释查询优化。简述查询优化规则和方法。

查询优化是利用等价变换对关系代数表达式进行优化，从而降低查询费用，提高查询效率。目标是选择高效查询策略，使用最低查询费用，实现快速数据查询。

查询优化规则：

（1）选择运算（减少元组数量）或者投影运算（减少属性个数）尽量先运算。

（2）选择和投影尽量同时运算。尽管有时需要先选择后投影，或者先投影后选择，如果可能，选择和投影尽量同时做，这样可以减少数据库的访问次数和对元组的扫描次数。

（3）投影运算尽量与其前后的双目运算同时运算（减少数据访问和扫描次数）。

（4）选择运算尽量与其前笛卡尔积合并为连接运算。连接运算通常比笛卡尔积生成更少的中间数据，从而节省更多时间。

（5）提取公共代数表达式（减少数据访问和扫描次数，避免重复运算）。对于多次重复使用的代数表达式，尽量提取出来，并保存其值作为中间结果。

查询优化方法：

（1）查询的关系代数表示。

（2）绘制初始查询树。

（3）分解选择和投影运算，更新查询树。

（4）合并选择和投影运算，更新查询树。

（5）结点分组，连接运算，优化查询树。

（6）生成查询优化器，提供查询优化方案。

2.11 写出如下查询的 SQL Server 语句和关系代数表达式，并绘制初始查询树和优化查询树；最后写出优化后的关系代数表达式。

查询在 2012 年 12 月 12 日之后（含当天）购买高等教育出版社出版的作者是王珊的图书的客户的姓名和电话。

（1）SQL Server 语句：

```
SELECT CName, Cust.Phone
FROM Cust, Buy, Book,Press
WHERE Cust.CNo=Buy.CNo AND Book.BNo=Buy.BNo AND
        Book.PNo=Press.PNo AND PDate>='2012-12-12' AND
        BName='高等教育出版社' AND Author='王珊'
```

（2）关系代数表达式：

$$\pi_{CName,Press.Phone}(\sigma_{PName='高等教育出版社'\wedge Author='王珊'\wedge PDate>='2012-12-12'\wedge Cust.CNo=Buy.CNo\wedge Book.BNo=Buy.BNo\wedge Book.PNo=Press.PNo}(Cust\times Buy\times Book\times Press))$$

（3）初始查询树、优化过程和最终生成的优化查询树如图2.1~2.3所示。

（4）优化后的关系代数表达式：

$$\pi_{CName,Cust.Phone}(\sigma_{Buy.BNo=Book.BNo}(\pi_{CName,Cust.Phone,Buy.BNo}(\sigma_{Cust.CNo=Buy.CNo}(Cust\times\sigma_{PDate>='2012-12-12'}(Buy))\times\pi_{Book.BNo}(\sigma_{Press.PNo=Book.PNo}(\sigma_{PName='高等教育出版社'}(Press)\times\sigma_{Author='王珊'}(Book))))))$$

即：

$$\pi_{CName,Cust.Phone}(\pi_{CName,Cust.Phone,Buy.BNo}(Cust\bowtie\sigma_{PDate>='2012-12-12'}(Buy))\bowtie\pi_{Book.BNo}(\sigma_{PName='高等教育出版社'}(Press)\bowtie\sigma_{Author='王珊'}(Book)))$$

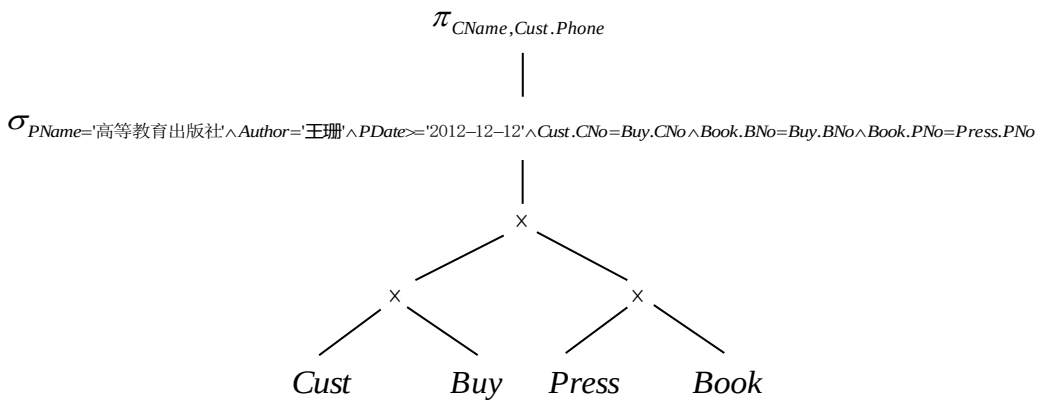


图 2.1　初始查询树

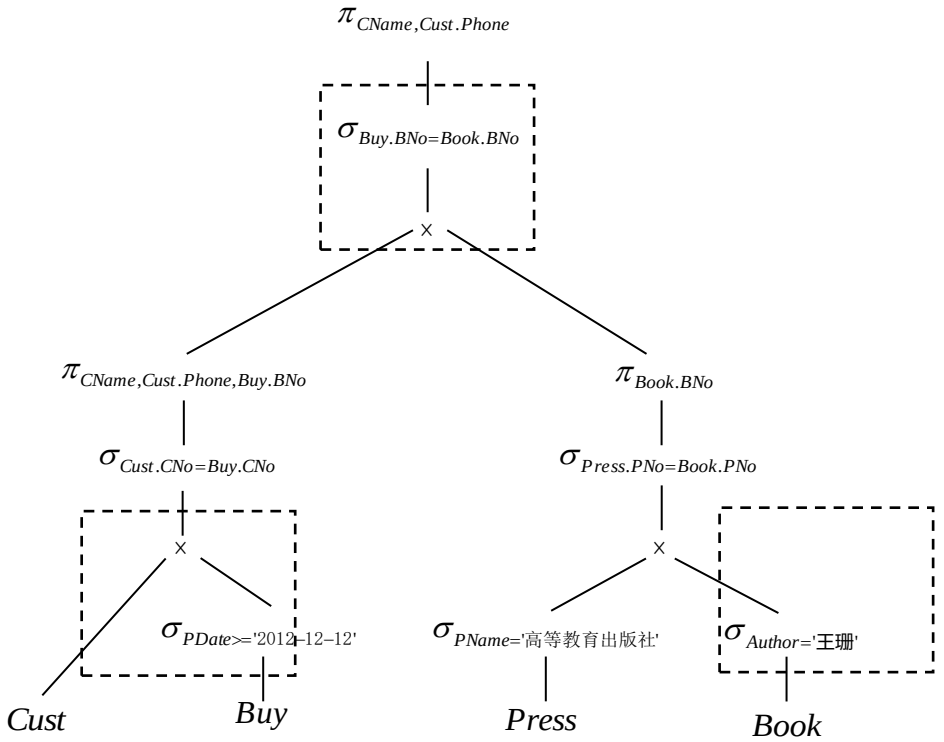


图 2.2　更新查询树

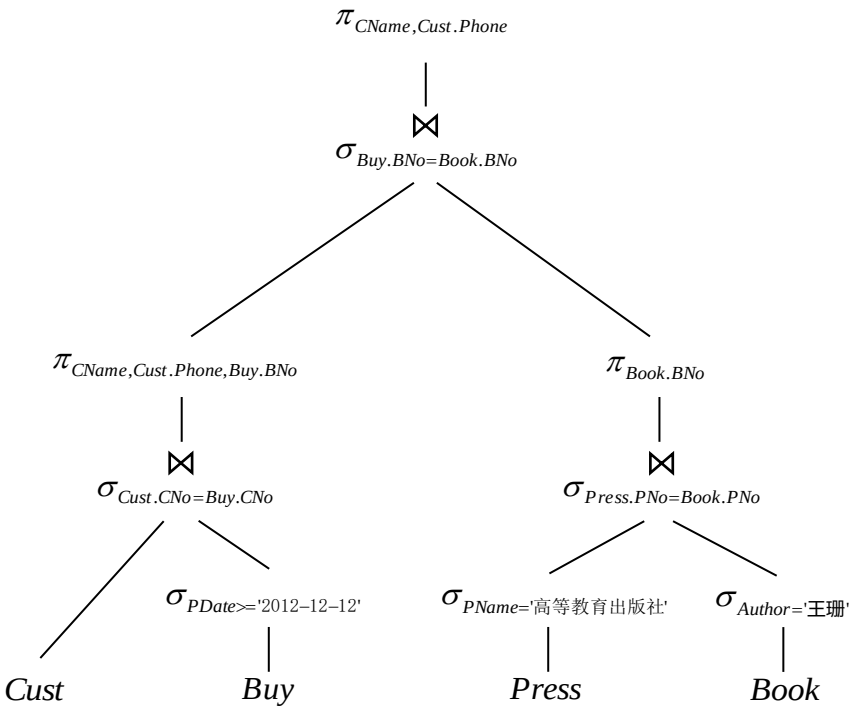

$\pi_{CName,Cust.Phone}$
$\bowtie$
$\sigma_{Buy.BNo=Book.BNo}$
$\pi_{CName,Cust.Phone,Buy.BNo}$
$\pi_{Book.BNo}$
$\bowtie$
$\sigma_{Cust.CNo=Buy.CNo}$
$\bowtie$
$\sigma_{Press.PNo=Book.PNo}$
$\sigma_{PDate>='2012-12-12'}$
$\sigma_{PName='高等教育出版社'}$
$\sigma_{Author='王珊'}$
Cust
Buy
Press
Book

图 2.3　优化查询树

第 3 章习题

3.1 解释外键、数据完整性和级联更新。

外键：已知 X 是关系 R 的主键，Y 是关系 S 的属性（或属性组），但不是 S 的主键，如果 Y 与 X 相对应，则称 Y 是关系 S 的外键。

数据完整性（数据库完整性）：指数据的正确性和相容性。

级联更新：对于违约操作，不是简单的拒绝，而是根据具体情况，把违约更新转换为非违约更新，然后接受更新。

3.2 简述数据完整性的内容及其约束规则。

数据完整性的内容：实体完整性、参照完整性和用户定义完整性等。

实体完整性规则：主属性 A 的取值不能为空值（NULL）。

参照完整性规则：如果 X 是关系 R 的主键，Y 是关系 S 的外键，且 Y 与 X 相对应，则 Y 的取值：要么为空值；要么取 X 的值。

用户定义完整性规则：用户根据系统需求自己定义的约束条件。

3.3 简述完整性控制机制的基本功能。

定义功能、检查功能和违约处理等。

3.4 简述完整性的违约处理方法。

拒绝更新、置空和级联更新等。

3.5 已知关系模式如下：

客户 C(客户号 CNo，客户名 CName，地址 CAddr)

产品 P(产品号 PNo，产品名 PName，价格 Price，供应商 PSupp)

订单 R(RNo，CNo，PNo，日期 RDate，数量 Quant)。

如果规定一张订单只能订购一种产品，请用 SQL Server 2016 语言创建上述关系模式，并完成以下完整性约束：

（1）定义相应的实体完整性。

（2）定义相应的参照完整性。

（3）定义价格不得高过 60（不含 60），且不得低于 20（含 20）。

```
-- 建立数据库 Cpr
 CREATE DATABASE Cpr
 GO
 --打开数据库 Cpr
 USE Cpr
 GO
 --建立客户表
```

```
CREATE TABLE C(
    CNo CHAR(6) PRIMARY KEY CHECK (CNo LIKE 'C[0-9][0-9][0-9][0-9][0-9]'),
    CName CHAR(8) NOT NULL,
    CAddr CHAR(22) NOT NULL)
--建立产品表
CREATE TABLE P(
    PNo CHAR(6) PRIMARY KEY CHECK (PNo LIKE 'P[0-9][0-9][0-9][0-9][0-9]'),
    PName CHAR(20) NOT NULL,
    Price DECIMAL(5,2) CHECK (Price>=20 AND Price<60),
    PSupp CHAR(26) NOT NULL)
GO
--建立订单表
CREATE TABLE R(
    RNo CHAR(6) PRIMARY KEY
        CHECK (RNo LIKE '[U-Z][0-9][0-9][0-9][0-9][0-9]'),
    CNo CHAR(6),
    PNo CHAR(6),
    RDate Date,
    Quant FLOAT,
    FOREIGN KEY (CNo) REFERENCES C(CNo)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (PNo) REFERENCES P(PNo)
        ON DELETE CASCADE ON UPDATE CASCADE)
```

3.6 已知关系供应商 S 和零件 P 如表 3.1 和表 3.2 所示，其主键分别是供应商号和零件号，P 的供应商号是 P 的外键，颜色只能取红、白或蓝。假设 DBMS 无级联功能。

表 3.1 供应商 S

供应商号	供应商名	城市
B01	红星	北京
S10	宇宙	上海
T20	黎明	天津
Z01	立新	重庆

表 3.2 零件 P

零件号	颜色	供应商号
010	红	B01
011	蓝	B01
201	蓝	T20
312	白	S10

（1）如果向 P 插入新元组（‘201’，‘白’，‘S10’）、（‘301’，‘红’，‘T11’）和（‘301’，‘绿’，‘B01’），则不能插入的元组是哪些？

（2）如果删除 S 的元组（‘S10’，‘宇宙’，‘上海’）和（‘Z01’，‘立新’，‘重庆’），则可以删除的是哪些？

（3）如果把 S 中供应商号的值‘Z01’改为‘Z30’，或者把 P 中供应商号的值‘T20’改为‘T10’，则可以执行的操作是哪些？

（4）S×P 的元组个数是多少。

（5）S 与 P 自然连接后的元组个数是多少？

（1）均不能。

（2）(‘Z01’，‘立新’，‘重庆’)。

（3）前者。

（4）16。

（5）4。

3.7 已知雇员关系和部门关系如表 3.3 和表 3.4 所示。

表 3.3 雇员

雇员号	雇员名	部门号	工资
001	张三	02	2000
010	王宏	01	1200
056	马林	02	1000
101	张三	04	1500

表 3.4 部门

部门号	部门名	电话	地址
01	业务部	000	A 楼
02	销售部	001	B 楼
03	服务部	002	C 楼
04	财务部	003	D 楼

（1）雇员的主键是哪些属性？

（2）雇员的外键是哪些属性？

（3）雇员名是雇员关系的候选键吗？

（4）如果部门关系中，财务部的部门号需要调整为 06，则雇员关系如何调整？

（5）把雇员马林调往服务部，则雇员关系如何调整？

（1）雇员号。

（2）部门号。

（3）错误。

（4）把部门号是 04 的改为 06。

（5）把马林的部门号是 02 的改为 03。

3.8 已知关系 R 和 S 如表 3.5 和 3.6 所示。

表 3.5 R

A	B	C
1	b1	c1
2	b2	c2
3	b1	c1

表 3.6 S

D	E	A
d1	e1	1
d2	e2	1
d3	e1	2

（1）定义 R 和 S 的完整性约束。

（2）在（1）的基础上，能否删除 R 中 A 的值分别为 2 和 3 的元组？说明原因。

（1）

CREATE TABLE R(A INT PRIMARY KEY,B CHAR(2),C CHAR(2))

```
CREATE TABLE S(D CHAR(2) PRIMARY KEY,E CHAR(2),A INT,
    FOREIGN KEY (A) REFERENCES R(A) ON DELETE RESTRICT)
```

（2）A 的值为 2 的元组不能删除，因为破坏了参照完整性的受限约束规则；A 的值为 3 的元组可以删除，语句为 DELETE FROM R WHERE A = 3。

第 4 章习题

4.1 简述 SQL Server 的特点。

综合统一、语法简单、易学易用、面向集合操作、高度非过程化和一语多用等。

4.2 解释 ODBC。简述 SQL 包含的子语言系统。

ODBC：Open Database Connectivity；开放数据库互连。数据库引擎数国际标准接口。

子语言系统：数据定义语言（Data Definition Language，DDL）；数据操纵语言（Data Manipulation Language，DML）；数据控制语言（Data Control Language，DCL）。

4.3 简述关系代数的选择、投影和连接，分别对应 SELECT…FROM…WHERE 的短语。

选择：在关系中选择出满足给定条件的元组；对应 WHERE。

投影：在关系 R 中选择出若干属性列组成的关系；对应 SELECT。

连接：把两个关系按照连接条件进行连接后生成一个新关系的过程；对应 FROM。

4.4 使用 SQL Server 创建习题 2.5 的 4 表。

```
CREATE DATABASE Spj
GO
USE Spj
CREATE TABLE S(
    SNo CHAR(6) PRIMARY KEY,
    SName CHAR(12) NOT NULL,
    City CHAR(20))
CREATE TABLE P(
    PNo CHAR(6) PRIMARY KEY,
    PName CHAR(20),
    Color CHAR(4),
    Weight INT)
CREATE TABLE J(
    JNo CHAR(6) PRIMARY KEY,
    JName CHAR(16) NOT NULL,
    City CHAR(20))
CREATE TABLE SPJ(
    SNo CHAR(6) REFERENCES S(SNo),
    PNo CHAR(6) REFERENCES P(PNo),
    JNO CHAR(6) REFERENCES J(JNo),
    Qty INT)
```

4.5 在习题 4.4 的 4 表中，使用 SQL Server 完成如下操作：

（1）查询所有供应商的姓名和所在城市。

（2）查询所有零件的名称、颜色、重量。

（3）查询使用供应商 S1 所供应零件的名称。

（4）查询能提供 P2 零件的供应商及其数量。

（5）查询上海厂商供应的所有零件号码。

（6）查询供应 S3 供应 P3 零件的名称。

（7）查询没有供应天津产的零件的供应商。

（8）把全部红色零件的颜色改成蓝色。

（9）删除供应商 S4，并从供应关系中删除其相应的记录。

（10）请将(S6，P6，300)插入到零件供应关系(SP)中。

```
SELECT SName,City FROM S
SELECT Pname,Color,Weight FROM P
SELECT PName FROM P,SP
    WHERE P.PNo=SP.PNo AND SNo='S1'
SELECT SNo,Qty FROM SP WHERE PNo='P2'
SELECT PNo FROM S,SP WHERE S.SNo=SP.SNo AND City='上海'
SELECT PName FROM P,SP
    WHERE P.PNo=SP.PNo AND SNo='S3' AND P.PNo='P3'
SELECT SNo FROM SP
EXCEPT
SELECT SP.SNo FROM S,SP
    WHERE S.SNo=SP.SNo AND City='天津'
UPDATE P SET Color='蓝色' WHERE Color='红色'
DELETE FROM S WHERE SNo='S4'
DELETE FROM SP WHERE SNo='S4'
INSERT INTO SP VALUES ('S6','P6',300)
```

4.6 已知数据库的关系分别为职工（工号，姓名，年龄，性别）、社团（团号，团名，团长，地点）、参加（工号，团号，日期）。使用 SQL Server 完成如下操作：

（1）查询参加篮球队的职工的工号和姓名。

（2）查询职工号为 666 的职工参加的社团名称。

（3）查询各个社团编号及相应的参加人数。

（4）建立视图 SInfo（工号，姓名，性别，年龄，团号，团名，团长，日期）。

（5）查询参加歌唱队或者篮球队的职工的工号和姓名。

（6）查询没有参加任何社团的职工基本信息。

（7）查询参加全部社团的职工基本信息。

（8）查询参加工号为 666 的职工所参加的全部社团的职工工号。

（9）统计每个社团的参加人数。

（10）查询参加人数最多的社团的团名和参加人数。

（11）查询参加人数超过 99 人的社团的团名和团长。

（12）把对职工和社团的查询和插入权限赋给李明，并允许其继续授权。。

```
（1）SELECT 工号，姓名 FROM 职工，参加，社团
        WHERE 职工.工号=参加.工号 AND 参加.编号=社团.编号 AND 名称='篮球队'
（2）SELECT 团名 FROM 参加 ,社团 WHERE 参加.团号=社团 .团号 AND 工号='666'
（3）SELECT 团号，COUNT(工号) FROM 参加 GROUP BY 团号
（4）CREAT VIEW SInfo AS
     SELECT 工号，姓名，性别，年龄，团号，团名，团长，日期
     FROM 职工,参加,社团 WHERE 参加.工号=职工.工号 AND 参加.团号=社团.团号
（5）SELECT 工号，姓名 FROM 职工，参加，社团
     WHERE 职工.工号=参加.工号 AND 参加.编号=社团.编号 AND
            社团.名称 IN（'歌唱队'，'篮球队'）
（6）SELECT * FROM 职工 WHERE NOT EXISTS（
        SELECT * FROM 参加 WHERE 参加.工号=职工.工号）
（7）SELECT * FROM 职工 WHERE NOT EXISTS（
        SELECT * FROM 参加 WHERE NOT EXISTS（
          SELECT *   FROM 社团
            WHERE 参加.工号=职工.工号 AND 参加.团号=社团.团号））
（8）SELECT 工号 FROM 职工 WHERE NOT EXISTS（
        SELECT * FROM 参加 X WHERE X.工号='999' AND NOT EXISTS（
          SELECT * FROM 参加 Y WHERE Y.团号=X. 团号 AND Y.工号=职工.工号））
（9）SELECT 团号，COUNT(工号) AS 人数 FROM 参加 GROUP BY 团号
（10）SELECT 团名，人数 FROM 社团 X，(SELECT 团号，MAX(人数）FROM
         (SELECT 团号，COUNT(工号) AS 人数 FROM 参加 GROUP BY 团号)) AS Y
            WHERE X.团号=Y.团号
（11）SELECT 团名，团长 FROM 社团 X，(SELECT 团号，COUNT(工号) AS 人数
         FROM 参加 GROUP BY 团号 HAVING COUNT(工号)>99) AS Y
           WHERE X.团号=Y.团号
（12）GRANT SELECT,INSERT ON 职工，社团 TO 李明 WITH GRANT OPTION
```

第 5 章习题

5.1 在 E-R 图中，如果有 10 个实体，9 个联系，其中 1:1、1:n 和 n:m 联系均为 3 个，则把 E-R 图转换为关系模型后，关系模式的理想个数是多少个。答案：13。

5.2 在蓝天物流数据库 BlueSky 中，需要管理供应商、项目、零件、仓库和职工的相关信息及其应用需求如下：

（1）供应商信息包括商号、商名、城市和电话等。

（2）项目信息包括项目号、项目名、单位和负责人等。

（3）仓库信息包括仓库号、面积和电话等。

（4）零件信息包括零件号、名称、规格和单价。

（5）职工信息包括工号、姓名、年龄和职称。

（6）一个供应商可以供应多个项目的多个零件，一个项目可以接收多个供应商的多个零件，接收供应的零件后，需要给出供应量。

（7）一个仓库可以存放多种零件，一种零件可以存放在多个仓库中，同时给出仓库存放零件的库存量。

（8）一个仓库聘用多个职工，一个职工只能在一个仓库工作，同时签约合同期限；职工之间具有领导与被领导关系，领导需要添加职务补助。

要求绘制 BlueSky 的 E-R 图，并转换为相应的关系模式，同时指明主键和外键。

答案：根据已知条件可知，E-R 图如图 5.1 所示。

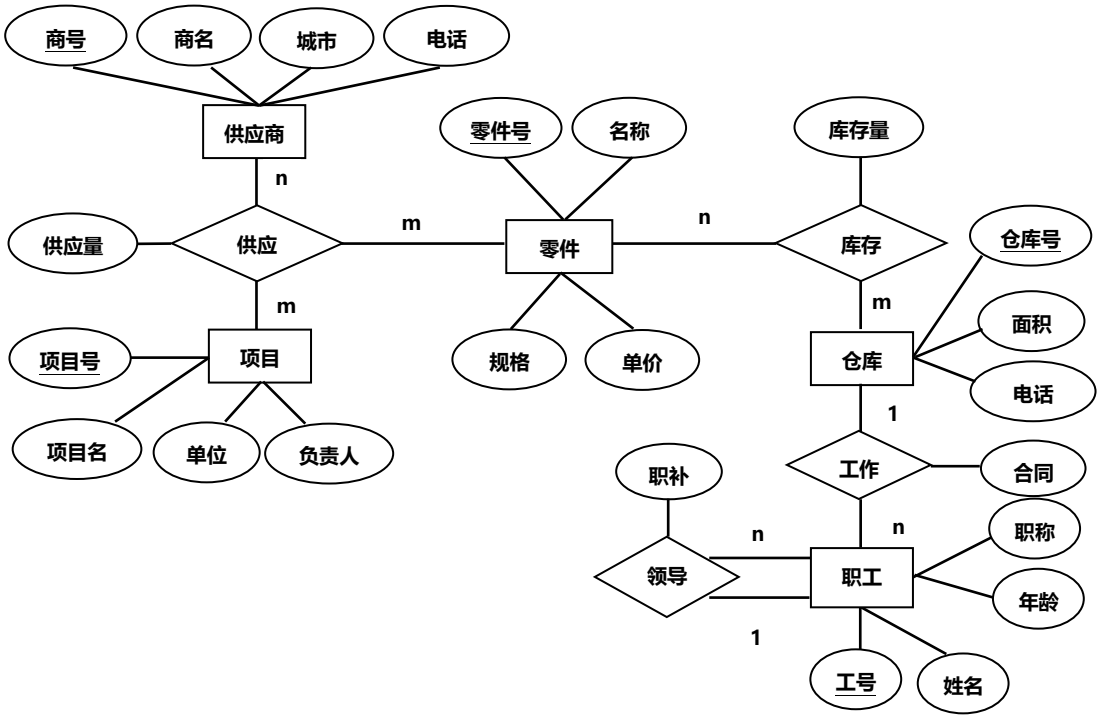


图5.1　物资管理E-R图

E-R 图转换为关系模式，并指明主键和外键。

综合分析：

（1）零件实体、仓库实体和职工实体分别转换为一个关系模式，转换后的键保持不变。转换的结果如下：

供应商（商号，商名，城市，电话）

主键：商号；外键：无。

仓库（仓库号，面积，电话）

主键：仓库号；外键：无。

零件（零件号，名称，规格，单价）

主键：零件号；外键：无。

职工（工号，姓名，年龄，职称）

主键：工号；外键：无。

项目（项目号，项目名，单位，负责人）

主键：项目号；外键：无。

（2）多对多联系，转换为一个独立关系模式。即：

供应（商号，项目号，零件号，供应量）

主键：（商号，项目号，零件号）；外键：商号，项目号，零件号。

库存（仓库号，零件号，库存量）；

主键：（仓库号，零件号）；外键：仓库号，零件号。

（3）工作一对多联系，可以转换为一个独立关系模式，也可以归并到多端实体。本例题采用归并到多端的方式。

职工（工号，姓名，年龄，职称，仓库号，合同）

主键：（工号）；外键：仓库号。

（4）领导一对多联系（自联系：同一实体内元组之间的联系），可以转换为一个独立关系模式，但是，由于与联系相关联的是同一实体，这样转换后的关系模式中就会出现相同的属性，但其所代表的意义不同。即：

领导（工号，工号，职补）

其中：前面的工号是主键，是多端职工的候选键“工号”；后面的工号虽然仍然是职工的“工号”，但是它的含义是领导者的“工号”（不妨设为：班长），因此，需要把后面的工号重新命名为“班长”。也就是说，班长属性的取值，仍然是工号。即：

领导（工号，班长，职补）

主键：（工号）；外键：班长。

领导联系也可以归并到多端实体，即把领导联系，归并到职工实体中。归并后的职工实体进一步调整如下：

职工（工号，姓名，年龄，职称，仓库号，合同，班长，职补）

主键：(工号)；外键：仓库号，班长。

根据上述分析，最终转换的关系模式如下：

供应商（商号，商名，城市，电话）

主键：商号；外键：无。

仓库（仓库号，面积，电话）

主键：仓库号；外键：无。

零件（零件号，名称，规格，单价）

主键：零件号；外键：无。

项目（项目号，项目名，单位，负责人）

主键：项目号；外键：无。

职工（工号，仓库号，姓名，年龄，职称，合同期，班长，职务补）

主键：(工号)；外键：仓库号，班长。

供应（商号，项目号，零件号，供应量）

主键：(商号，项目号，零件号)；外键：商号，项目号，零件号。

库存（仓库号，零件号，库存量）；

主键：(仓库号，零件号)；外键：仓库号，零件号。

5.3 在运输公司数据库 TranInfo 中，需要管理车队、车辆和司机的基本信息。即：

（1）车队信息包括车队号、车队名和人数等。

（2）车辆信息包括车号、厂家和出厂日期等

（3）司机信息包括司机号、姓名和电话等。

要求：车队与司机之间存在聘用联系，每个车队可聘用多个司机，但是每个司机只能应聘于一个车队，车队聘用司机有个聘期；车队与车辆之间存在拥有联系，每个车队拥有多辆汽车，但是每辆车只能属于一个车队；司机与车辆之间存在着驾驶联系，司机驾驶车辆有驾驶里程，每个司机可以驾驶多辆汽车，每辆汽车可以被多个司机驾驶。

完成如下任务：

（1）绘制 TranInfo 的 E - R 图。

（2）把 E - R 图转换成关系模型，并标注主键和外键。

答案：根据已知条件可知，E-R 图如图 5.2 所示。

E-R 图转换为关系模式，并指明主键和外键。

车队（车队号，车队名，人数）

主键：车队号；外键：无。

车辆（车号，厂家，生产日期，车队号）

主键：车号；外键：车队号。

司机（司机号，姓名，电话，车队号，聘期）

主键：司机号；外键：车队号。

驾驶（司机号，车号，里程）

主键：(司机号，车号)；外键：车队号，车号。

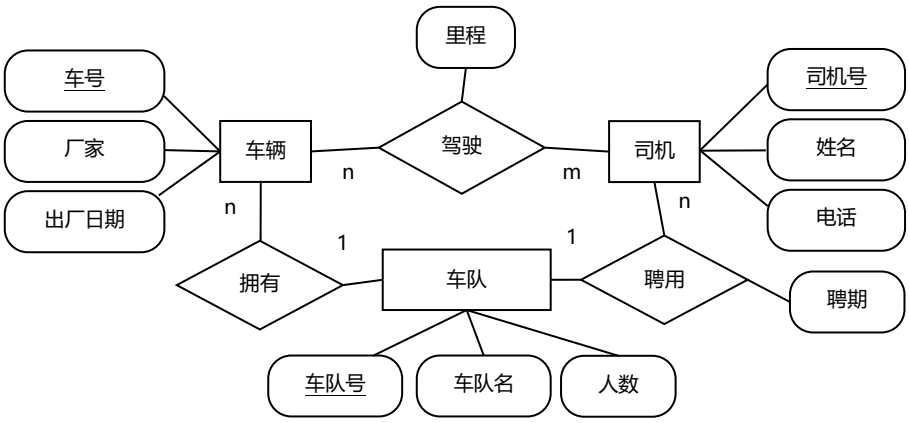


图5.2　TranInfoE-R图

5.4 在商品销售数据库 ComInfo 中，需要管理商品、商店和职工的相关信息为品号、品名、厂商、厂址、单价、数量；店号、店名、店址、电话；工号、姓名、性别、年龄、电话等。同时要求一个商店可以销售多个商品，一个商品可以在多个商店销售，同时需要给出销量；一个职工只能在一个商店工作，一个商店可以有多个职工，同时需要给出工资和聘期。职工之间存在领导关系。设计 ComInfo 的概念模型和逻辑模型。

答案：根据已知条件可知，概念模型如图 5.3 所示。

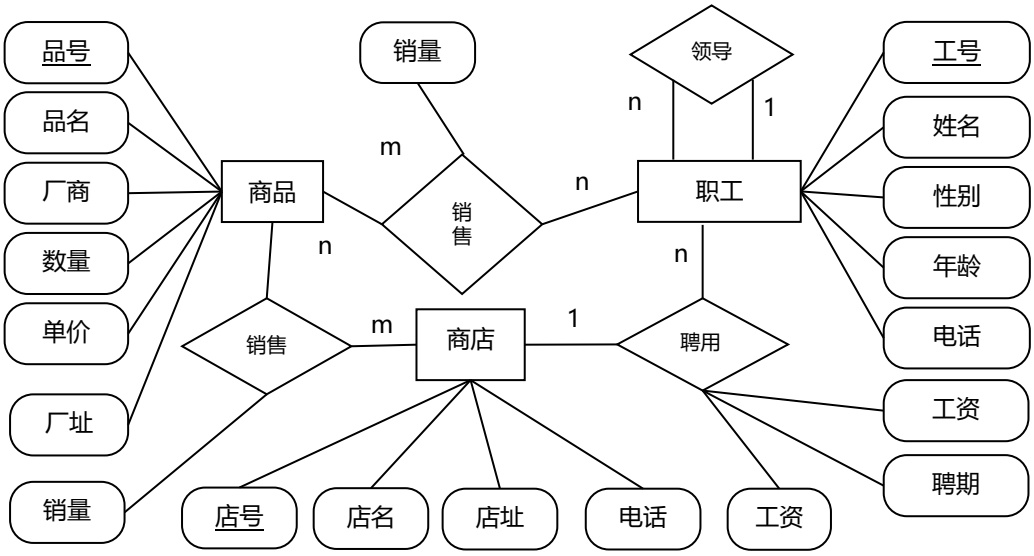


图5.3　ComInfoE-R图

逻辑模型：

商品（品号，品名，厂商，厂址，单价，数量）

主键：品号；外键：无。

商店（店号，店名，店址，电话）

主键：店号；外键：无。

职工（工号，姓名，性别，年龄，电话，店长，店号，工资，聘期）

主键：工号；外键：店长（取值同工号），店号。

销售（店号，品号，销量）

主键：(店号，品号)；外键：店号，品号。

5.5 在图书流通数据库 CirBook 中，用于管理学生、管理员、图书和书库的流通业务，要求满足学生借阅图书、书库存放图书、管理员管理书库中的图书等。

建立 CirBook 的概念模型（E-R 图）和逻辑模型，并标注主键和外键。

答案：根据已知条件可知，概念模型如图 5.4 所示。

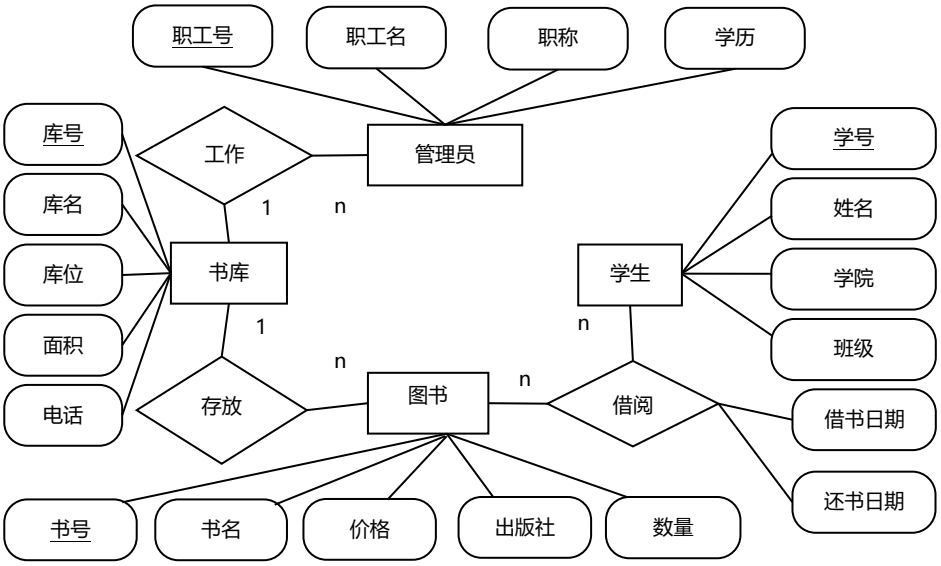


图5.4 CirInfoE-R图

逻辑模型：

图书（书号，书名，价格，出版社，数量，库号）

主键：书号；外键：库号。

学生（学号，姓名，学院，班级）

主键：学号；外键：无。

书库（库号，库名，库位，面积，电话）

主键：库号；外键：无。

管理员（职工号，职工名，职称、学历，库号）

主键：职工号；外键：库号。

借阅（学号，书号，借书日期，还书日期）

主键：(学号，书号)；外键：学号，品号。

5.6 熊猫集团拥有多个连锁商场，需要构建一个数据库系统，管理如下相关业务：

（1）商场需要记录的信息包括商场编号（不能重复）、商场名称、地址和联系电话。商场信息如表 5.1 所示。

表 5.1　商场信息表

商场编号	商场名称	地址	联系电话
PS2101	淮海商场	淮海中路 918 号	021-64158818
PS2902	西大街商场	西大街时代盛典大厦	029-87283220
PS2903	东大街商场	碑林区东大街 239 号	029-87450287
PS2901	长安商场	雁塔区长安中路 38 号	029-85264953

（2）每个商场包含不同的部门，部门需要记录的信息包括部门编号（不同商场的部门编号不同）、部门名称、位置分布和联系电话。商场的部门信息如表 5.2 所示。

表 5.2　部门信息表

部门编号	部门名称	位置分布	联系电话
DT002	财务部	商场大楼六层	82504342
DT007	后勤部	商场地下副一层	82504347
DT021	保安部	商场地下副一层	82504358
DT005	人事部	商场大楼六层	82504446
DT001	管理部	商场大楼三层	82504668

（3）每个部门雇用了多名员工处理日常事务，每名员工只能属于一个部门。员工需要记录的信息包括员工编号、姓名、岗位、电话号码和工资。员工信息如表 5.3 所示。

表 5.3　员工信息表

员工编号	姓名	岗位	电话号码	工资
XA3310	周超	理货员	13609257638	1500.00
SH1075	刘飞	防损员	13477293487	1500.00
XA0048	江雪花	广播员	15234567893	1428.00
BJ3123	张正华	经理	13345698432	1876.00

要求完成如下任务：

（1）根据需求分析阶段收集的信息，设计系统的 E-R 图。

（2）把系统的 E-R 图转化为理想的关系模式，并标注主键和外键。

答案：根据已知条件可知，概念模型如图 5.5 所示。

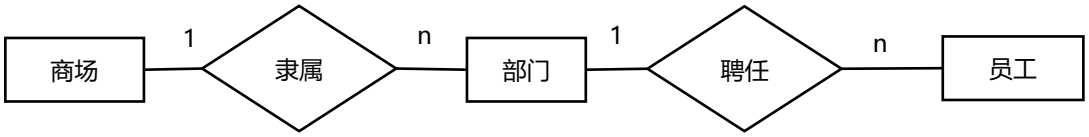


图5.5　系统E-R图

逻辑模型：

商场（商场编号，商场名称，地址，联系电话）

　　主键：商场编号；外键：无。

部门（部门编号，部门名称，位置分布，联系电话，商场编号）

　　主键：部门编号；外键：商场编号。

员工（员工编号，姓名，岗位，电话号码，工资，部门编号）

　　主键：员工编号；外键：部门编号。

第 6 章习题

6.1 解释函数依赖、完全函数依赖、部分函数依赖和传递函数依赖。

函数依赖：设 R(U)是属性集 U 上的关系模式，X 和 Y 是 U 的子集。对于 R(U)的任意关系的任意元组 t_1, t_2，如果 $t_1[Y]\neq t_2[Y]$，成立 $t_1[X]\neq t_2[X]$（即：不存在 t_1, t_2，使得 $t_1[X]=t_2[X]$，而 $t_1[Y]\neq t_2[Y]$）。则 Y 函数依赖于 X。

完全函数依赖：在 R(U，F)中，如果 X→Y，且 Y 不函数依赖于 X 的任意真子集。则 Y 完全函数依赖于 X。

部分函数依赖：在 R(U，F)中，如果 X→Y，且 Y 不完全函数依赖于 X。则 Y 部分函数依赖于 X。

递函数依赖：在 R(U，F)中，如果 X→Y，Y→Z，并且 Y $\not\rightarrow$ X。

则称 Z 传递函数依赖于 X。

6.2 给出候选键三个不同的定义。

候选键 CK：对于指定的关系，能够区分每一个元组的最小属性集。

候选键 CK：对于 R(U，F)，如果 X ⊆ U，且 $X \xrightarrow{F} U$，则 X 是候选键。

候选键 CK：对于 R(U，F)，如果 X⊆ U，且 X_F^+ =U，则 X 是候选键。

6.3 解释范式，简述常用的范式及其之间的包含关系。

范式：满足系统规范要求的关系模式的集合。即：规范化的关系模式的集合。

常用范式：1NF、2NF、3NF、BCNF、4NF 和 5NF。

包含关系：1NF ⊃ 2NF ⊃ 3NF ⊃ BCNF ⊃ 4NF ⊃ 5NF。

6.4 解释关系模式规范化。简述关系模式的规范化过程。

关系模式规范化：利用关系规范化理论，对低级的关系模式进行分解和优化，从而消除冗余的函数依赖，使其达到系统需要的更高的规范要求，并在一定程度上解决插入异常、修改异常、删除异常和数据冗余等问题的过程。即：分解低级关系模式使其转换为高级关系模式的过程(范式分解)。

关系模式的规范化过程：

1NF → 消除非主属性对候选键的部分函数依赖 → 2NF

2NF → 消除非主属性对候选键的传递函数依赖 → 3NF

3NF → 消除主属性对候选键的部分和传递函数依赖 → BCNF

BCNF → 消除非平凡且非函数依赖的多值依赖 → 4NF

4NF → 消除连接依赖 → 5NF

6.5 解释范式分解。简述保连接分解和保依赖分解。

范式分解：分解低级范式，并转换为高级范式的过程。

保连接分解：保持关系模式下关系的元组信息不发生改变的分解。即：

已知关系模式 R(U,F)的一个分解 $\mathscr{R}$ ={$R_1(U_1,F_1)$，$R_2(U_2,F_2)$，…，$R_n(U_n,F_n)$}，$R_1(U_1,F_1)$，$R_2(U_2,F_2)$，…，$R_n(U_n,F_n)$的连接与 R(U,F)等价，亦即：

$$R(U,F) \equiv \bowtie_{i=1}^{n} R(U_i,F_i)$$

保依赖分解：保持关系模式的函数依赖关系不发生改变的分解。即：

已知关系模式 R(U,F)的一个分解 $\mathscr{R}$={$R_1(U_1,F_1)$，$R_2(U_2,F_2)$，…，$R_n(U_n,F_n)$}，则($F_1 \cup F_2 \cup \ldots \cup F_n$)≡F，亦即：

$$F^+ = \left(\bigcup_{i=1}^{n} F_i\right)^+$$

6.6 证明：Armstrong 公理的推广规则：

（1）合并律：如果 X→Y，X→Z，则 X→YZ。

（2）分解律：如果 X→YZ，则 X→Y，X→Z。

（3）伪传递：如果 X→Y，YW→Z，则有 XW→Z。

证明：

（1）

∵ X → Y；根据增广律，成立 XX → XY；∴ X → XY

∵ X → Z；根据增广律，成立 XY → ZY

∴ 根据传递律，成立 X → YZ

（2）

∵ Y ⊆ YZ；根据自反律，成立 YZ → Y

∵ Z ⊆ YZ；根据自反律，成立 YZ → Z

∵ X →YZ；根据传递律，成立 X→Y 和 X→Z

（3）

∵ X → Y；根据增广律，成立 XW → YW

又∵ YW → Z；根据传递律，成立 XW → Z

6.7 证明：X→$A_1A_2\ldots A_k$ 当且仅当 X→A_i(i=l,2,…,k)。

证明：使用数学归纳法。

（1）当 n=1 时，显然成立：X→A_1 当且仅当 X→A_1。

（2）假设当 n=k 时成立，即：X→$A_1A_2\ldots A_k$ 当且仅当 X→A_i(i=l,2,…,k)

（3）当 n=k+1 时成立，证明 X→$A_1A_2\ldots A_{k+1}$ 当且仅当 X→A_i(i=l,2,…,k+1)

1）首先证明：若 X→$A_1A_2\ldots A_{k+1}$，则 X→A_i(i=l,2,…,k+1)

∵ X→$A_1A_2\ldots A_{k+1}$；∴ 根据分解律，成立 X→$A_1A_2\ldots A_k$ 和 X→A_{k+1}；

根据假设，由 X→$A_1A_2\ldots A_k$ 成立，可以导出 X→A_i(i=l,2,…,k)成立；

∴ $X \to A_i(i=l,2,\ldots,k+1)$成立。

2）其次证明：若 $X \to A_i(i=l,2,\ldots,k+1)$ ，则 $X \to A_1A_2\ldots A_{k+1}$

根据假设，由 $X \to A_i(i=l,2,\ldots,k)$成立，可以导出 $X \to A_1A_2\ldots A_k$ 成立；

根据合并律，由 $X \to A_1A_2\ldots A_k$ 和 $X \to A_{k+1}$，可以导出 $X \to A_1A_2\ldots A_{k+1}$ 成立。

6.8 对于 R(U,F)，U={A,B,C,D}，F={A→B,B→A,B→C,A→C,C→A}，计算 F_{min}。

答案：

方法 1：

∵ A→B,B→C；∴ A→C 冗余。

∵ B→C,C→A；∴ B→A 冗余。

∴ F_{min}={A→B,B→C,C→A}。

方法 2：

∵ B→A,A→C；∴ B→C 冗余。

∴ 或者 F_{min}= {A→B，B→A，A→C，C→A}。

6.9 对于 R(U,F),U={A,B,C,D,E}，F={AB→C,C→D,D→E}，如下分解是否保连接？

$\mathscr{R}$ {R1(A,B,C),R2(C,D),R3(D,E)}。

答案：根据保连接分解“算法 6.3”可知：该分解是保连接分解。

6.10 对于 R(U,F),U={A,B,C,D,E}，F={A→C,B→C,C→D,DE→C,CE→A},判断：

$\mathscr{R}${R1(A,D),R2(A,B),R3(B,E),R4(C,D,E),R5(A,E)}是否保连接？

答案：根据保连接分解“算法 6.3”可知：该分解是保连接分解。

6.11 如果 $\mathscr{R}$ {R1(U1,F1),R2(U2,F2)}是 R(U,F)的分解，证明：

$\mathscr{R}$ 保连接 当且仅当 U1∩U2→U1-U2 或者 U1∩U2→U2-U1。

提示：把 U 分解成 U1∩U2、U1-U2 和 U2-U1，然后利用保连接分解算法证明即可。

证明：构造分解状态表如下（a 和 b 的下标省略）：

表 6.1 分解状态表

	U1∩U2	U1-U2	U2-U1
U1	a…a	a…a	b…b
U2	a…a	b…b	a…a

（1）充分性：

如果 U1∩U2→U1-U2 在 F 中，则根据保连接算法，可以把第 2 行中对应 U1-U2 列的 b…b 改成 a…a，从而使得第 2 行为全 a，因此该分解保连接。

同理：如果 U1∩U2→U2-U1 在 F 下成立，则根据保连接算法，可以把第 1 行中对应 U2-U1 列的 b…b 改成 a…a，从而使得第 1 行为全 a，因此该分解保连接。

另外：如果 U1∩U2→U1-U2 不在 F 中，但是在 F+中（即：在 F 下成立），则根据 Armstrong 公理，成立 U1∩U2→Ai，而 Ai ⊆ U1-U2，所以第 2 行中 Ai 列的 b 改为 a，同样可以把 U1-U2

中的其它属性列的 b 也改为 a，从而使得第 2 行为全 a，因此该分解保连接。

同理：可以证明 U1∩U2→U2-U1 的情况。

（2）必要性：

如果该分解保连接，则构造的状态表 6.1 中，至少有一行为全 a。

如果第 1 行为全 a，则根据函数依赖的定义可知，成立：U1∩U2→U2-U1。

如果第 2 行为全 a，则根据函数依赖的定义可知，成立：U1∩U2→U1-U2。

6.12 对于 R(U,F),U={A,B,C,D,E,P},F={A→B,AE→P, CD→A,CE→D, BC→D} 则给出 R(U,F)的一个保连接保依赖分解。

答：根据算法，可知：{R1(A，B)，R2(A，E，P)，R3(C，D，A)，R4(C，E，D)，R5(B，C，D)}是保连接保依赖分解。

6.13 对于 R(U,F),U={A,B,C,D,E,G},F={B→A,B→D,C→G,C→BE,G→B,E→G}。则：

（1）计算 R 的所有候选键。

（2）计算 Fmin。

（3）判断 R∈3NF？如果满足，则说明原因；否则进行 3NF 分解。

答案：

（1）候选键 C。原因是 C 的闭包等于 U。

（2）最小等价函数依赖集：{B→A，B→D，C→E，G→B，E→G}

（3）不是，原因是存在传递依赖。分解结果如下：

R1（{B，A，D}，{ B→A，B→D}）

R2（{C，E}，{C→E}）

R3（{G，B}，{G→B}）；R4（{E，G}，{E→G}）

6.14 对于 R(U,F)，U={A,B,C,D,E}，F={A→D,E→D,D→B,BC→D,DC→A,B→AD}。

（1）计算 R 的所有候选键。计算 Fmin。

（2）判断 R∈3NF？如果满足，则说明原因；否则进行 3NF 分解。

答案：

（1）候选键 CE。原因是 CE 的闭包等于 U。

最小函数依赖集 Fmin：{A→D，E→D，D→B，B→A}

（2）不是，原因是存在传递依赖。分解结果如下：

R1（{A，D}，{ A→D}）

R2（{A，B}，{B→A}）

R3（{B，C，E}，{E→B}）

6.15 对于 R(U,F) ,U={A,B,C,D,E ,G} ,F={A→C,C→A,B→AC,BD→AE,D→AC,E→A}。则：

（1）计算 R 的所有候选键。计算 Fmin。

（2）在 F 下，B→DE？

答案：

（1）BDG。原因是 BDG 的闭包等于 U。

最小函数依赖集 Fmin：{A→C，C→A，B→A，BD→E，D→A，E→A}。

（2）无法推出。B→DE 不属于 F 的闭包。

6.16 对于 R(U,F)，U={A,B,C,D,E,G}，F={AB→C,C→A,BC→D,ACD→B,D→EG,BE→C,CG→BD,CE→AG}。则：

（1）计算 AB 和 BD 的闭包。判断 AB 和 BD 是否为 R 的候选键？请说明原因。

（2）判断 R 是否满足 3NF 和 BCNF？说明原因。

答案：

（1）AB 和 BD 的闭包均是{ABCDEG}。

是，原因是 AB 和 BD 的闭包均等于 U。

（2）是 3NF，原因是 F 中不存在非主属性对候选键的部分依赖和传递依赖。不是 BCNF，原因在 F 中存在非平凡函数依赖不包含候选键。

6.17 对于 R(U,F)，U={A,B,C,D,E,G}，F={AB→E,CD→G,B→D,C→A,D→B,G→AD}。则：

（1）计算 R 的所有候选键。计算 AB 的闭包。

（2）判断 R∈3NF？如果满足，则说明原因；否则进行 3NF 分解。

答案：

（1）CD、CB、CG，原因是它们的闭包均等于 U。

（2）AB 的闭包等于{ABDE}。

（3）不是 3NF，原因是 F 中存在非主属性对候选键的部分依赖和传递依赖。分解如下：

R1（{A，B，E}，{AB→E}）；

R2（{C，D，G}，{CD→G}）

R3（{B，D }，{B→D, D→B}）

R4（{C，A }，{C→A }）

R5（{G，A，D }，{G→AD}）

6.18 对于 R(U,F)，U={A,B,C}，F={A→B,B→C}，判断如下三组分解的保连接性。

（1）R1{AB，AC}

（2）R2{AB，BC}

（3）R3{AC，BC}

答案：R1，R2 保连接（即：无损连接）；R3 不保连接（即：有损连接）。

6.19 对于 W(C,P,S,G,T,R)，D={C→P,SC→G,TR→C,TP→R,TS→R}。其中 C－课程，P－教师，S－学生，G－成绩，T－时间，R－教室。

（1）计算 W 的一个候选键。判断 W 满足的最高范式。

（2）如果 W 分解为 3 个关系模式 W1(C,P)，W2(S,C,G)，W3(S,T,R,C)，判断 W1、W2、

W3 满足的最高范式。

答案：

（1）TS，原因是它们的闭包均等于 U。

根据规范理论可知：W 的规范化程度最高达到 2NF。

（3）根据规范理论可知：W1、W2、W3 的规范化程度分别最高达 BCNF；BCNF；2NF。

第 7 章习题

7.1 解释数据安全标准和数据安全模型。简述TDI和TCSEC按照哪几个等级和哪几个方面，详细描述了计算机系统安全的具体指标。

数据安全标准：《可信计算机系统评估准则》和《可信数据库解释》等。

《可信计算机系统评估准则》TCSEC:Trusted Computer System Evacuation Criteria。

《可信数据库解释》TDI:Trusted Database Interpretation。

TDI 和 TCSEC 按照四组七个等级（D（最小保护）；C1（自主存取保护），C2（受控存取保护）；B1（强制存取保护），B2（结构化保护），B3（安全域）；A1（验证设计）），从安全策略、责任、保证和文档等，详细描述了数据安全的具体指标。

7.2 解释计算机系统安全。简述计算机系统安全的具体内容。

计算机系统安全：为了保护计算机系统的硬件、软件及其数据，避免遭到偶然或自然、无意或有意的破坏或泄露，而采取的保护措施。

安全内容：技术安全（核心）、管理安全和政策法律等。

7.3 解释数据安全。

数据（库）安全：保护数据库防止非法使用造成的数据泄露、更改或破坏等。

7.4 解释数据存取控制。简述常用的数据存取控制方法。

数据存取控制：在存取数据时，需要确定用户是否拥有访问数据的权限，从而确保授权用户对数据的合法访问。

存取控制方法：自主存取控制和强制存取控制等，使产品达到 C2 级和 B1 级。

7.5 简述 SQL Server 2016 提供的权限控制语句。

授权 GRANT、阻权 DENY 和收权 REVOKE 等。

7.6 简述 SQL Server 2016 提供的两种身份验证方式。

Windows 身份验证：定义了一个存在于操作系统层次上的用户账户。

SQL Server 身份验证则：在 SQL Server 内部中创建的一个登录，并且与口令有关(SQL Server 登录与现有的 Windows 用户账户可能相同)。

7.7 简述 SQL Server 2016 提供的两个安全子系统。

Windows 安全系统：在操作系统层次上的安全系统。即：用户使用他们 Windows 用户账户(User Account)连接到 Windows 的方式。

SQL Server 安全系统：SQL Server 系统层次上所必需的附加安全系统。即：已经登录到操作系统的用户随后该如何连接到 SQL Server。

7.8 简述 SQL Server 2016 提供的两种身份验证模式。

Windows 模式：只使用 Windows 用户账号来登录 SQL Server 系统。SQL Server 介绍用户账号，并假设它在操作系统层次上是合法的。因为 SQL Server 相信操作系统已经确认了

账号和相应口令的合法性，所以这种连接到数据库的方式称为信任连接(Trusted Connection)。

Mixed 模式：允许用户使用 Windows 身份验证或 SQL Server 身份验证来连接到 SQL Server。这就意味着有些用户账户可以设置为使用 Windows 安全子系统，而另一些用户则可以使用 Windows 安全子系统之外的 SQL Server 安全子系统。

7.9 简述 SQL Server 2016 的登录用户，数据库用户和数据库角色。

登录 SQL Server 的用户首先必须是 Windows 合法用户或者 SQL Server 的合法用户。然后如果用户需要继续访问系统中的数据库，还必须拥有一个数据库用户账户。因此，对于每一个需要访问的数据库，用户都必须拥有该数据库的账户。数据库用户账户可以从现有的 Windows 用户账户、Windows 组(用户是该组的成员)、SQL Server2016 登录或者角色中映射过来。

登录用户：登陆 SQL Server 服务器的合法用户。

数据库用户：从 Windows 用户或者 SQL Server 登录映射过来的可以访问指定数据库的合法用户。

数据库角色：为了更好管理数据库用户，为多个同类的数据库用户创建的一个角色。

7.10 解释视图和表。简述视图与表的关系。简述视图的作用。

视图：利用查询语句定义的，从一个或者多个表中导出的虚表。

表（关系）: 笛卡尔积的子集。表（关系）=表结构（关系模式）+记录集合（元组集合）。

视图与表的区别：

（1）概念不同。表是使用 DDL 定义的，以独立文件的形式存储在外存上的真实数据。而视图则是利用查询语句定义的，从表或视图中导出的虚表。

（2）数据有效期不同。表是拥有真实数据的实表，永远有效；而视图则是只有查询语句的没有数据的虚表，在执行视图时，才临时有效。

（3）数据存储不同。表永久存储在外存，而视图中没有数据。

（4）更新方式不同。表可以任意更新，而视图不能任意更新，而是有条件更新。

视图与表的联系：

视图最终定义在表之上，视图数据来自于表；表是视图的基础，并为视图提供数据，如果表的结构发生了改变，或者删除了表，则相应的视图就会失去意义，因此需要修改或者删除相应的视图。

对于最终用户来说，视图与表等价。即：视图和表在用户看来都是“表”，用户完全可以按照表的使用方法来使用视图，同时可以在视图上定义视图。

视图作用：

（1）简化用户操作。

（2）清晰表达查询。

（3）同一数据可以以不同的形式提供给不同用户。

（4）在一定程度上确保了数据安全。

（5）在一定程度上提供了逻辑独立性。

7.11 解释审计和数据加密。

审计：用于监视并记录对数据库的操作行为，实时智能地解析对数据库的各种操作，并记入审计日志，以便日后进行查询、分析和过滤，实现对用户操作的监控和审查。

数据加密：根据预定的算法将原始数据（明文）变换为不可直接识别的结果数据（密文）的过程。

第 8 章习题

8.1 解释事务。简述事务的四个特性。

事务：用户定义的一个数据库操作序列，要么全做，要么不做。

特性：原子性、一致性、隔离性和持续（永久）性等。即：ACID 特性。

8.2 解释并发控制。简述并发控制与数据共享的关系。

并发控制：为了确保 ACID 特性，而对并发事务采取的封锁和调度等控制技术。

并发控制与数据共享的关系：

（1）数据共享和并发控制的定义。

（2）共享会导致并发，并发可能破坏完整性，从而需要进行并发控制。

（3）并发控制级别越高，其共享度越低；并发控制范围越大，其共享度越低。

（4）控制范围为数据库、关系、字段、记录等。

（5）控制级别为利用 X 锁和 S 锁组成封锁协议。

8.3 简述并发事务导致的数据不一致。如何解决？

并发事务导致的数据不一致：丢失修改、读脏数据和不可重读等。

解决方法：一级封锁协议解决丢失修改，二级封锁协议解决读脏数据，三级封锁协议解决不可重读等。

8.4 解释封锁。简述常用的封锁。

封锁：事务 T 在对数据对象 D 操作之前，先对 D 加锁 L，并获得对 D 的控制权限，在 T 释放 L 之前，其它事务不能更新 D。

常用封锁类型：排它锁和共享锁等。

8.5 解释封锁协议？简述常用的封锁协议。

封锁协议：为了解决并发事务所导致的数据不一致，利用 X 锁、S 锁和 I 锁等对数据对象加锁时，所规定的封锁规则。

常用封锁协议：一级封锁协议、二级封锁协议、三级封锁协议和两段锁协议等。

8.6 解释并发事务的串行调度和并行调度。简述二者的关系。

并发事务的串行调度：针对并发事务，按照一定的顺序依次执行每一个事务。

并发事务的并行调度：针对并发事务，同时执行若干个事务。

二者的关系：

（1）并发事务在串行执行过程中，事务之间不会产生相互影响，尽管并发事务不同的串行调度的执行结果可能不同，但不会导致数据不一致。因此并发事务的串行调度是正确调度。

（2）并发事务在并行执行过程中，事务之间可能发生相互影响，因此并发事务的并行调度可能是错误调度。

（3）因为并发事务的串行调度总是正确的，所以可以使用并发事务的串行调度来验证并

发事务的并行调度的正确性。

8.7 解释并发事务的可串行化。简述两段锁协议与并发事务可串行化的关系。

并发事务的可串行化：如果并发事务的并行调度的执行结果与并发事务的某个串行调度的执行结果相同。

如果并发事务的并行调度遵守两段锁协议，则并发调度可串行化。

如果并发事务的并行调度不遵守两段锁协议，则并发调度也可能可串行化。即：两段锁协议是并发事务可串行化的充分条件，而非必要条件。

8.8 如果 A=6，B=6，并发事务 T1，T2，T3 如下：

T1：A ← A+B；T2：B ← A-B；T3：A ← A*B。

完成如下操作：

（1）给出 T1，T2，T3 的所有串行调度及其执行结果。

（2）给出 T1，T2，T3 满足两段锁协议的可串行化并行调度及其执行结果。

（3）T1，T2，T3 是否存在不满足两段锁协议的可串行化并行调度？如果存在，给出相应的可串行化并行调度及其执行结果。

（4）如果 T1，T2，T3 满足两段锁协议，是否存在形成死锁的？如果存在，给出相应的调度。

答案：

（1）方案 1～方案 6 及其执行结果：

T1→T2→T3；A=72，B=6。T1→T3→T2；A=72，B=66。T2→T1→T3；A=0，B=0。T2→T3→T1；A=0，B=0。T3→T1→T2；A=42，B=36。T3→T2→T1；A=66，B=30。

（2）两段锁并发事务可串行调度策略如表 8.1 所示。

（3）非两段锁并发事务可串行调度策略如表 8.2 所示。

表 8.1 两段锁并发事务可串行调度

T_1 事务	T_2 事务	T_3 事务
Xlock A		Slock B
R(A)=6		R(B)=6
Slock B		Xlock A
R(B)=6	Xlock B	等待
A+B=12	等待	等待
W(A)=12	Slock A	等待
Unlock B	等待	Unlock B
Unlock A	R(B)=6	等待
	R(A)=12	等待
	W(B)=A-B	等待
	Unlock A	等待
	Unlock B	等待
		R(A)=12
		W(A)=A*B=72
		Unlock A

注：R(A)表示读取 A，W(A)表示写入 A

表 8.2 非两段锁并发事务可串行调度

T_1 事务	T_2 事务	T_3 事务
Slock B		
R(B)=6	Slock B	
Unlock B	R(B)=6	Slock B
Xlock A	Unlock B	R(B)=6
R(A)=6	Slock A	Unlock B
A=A+B	等待	Xlock A
W(A)=12	等待	等待
Unlock A	等待	等待
	R(B)=12	等待
	Unlock A	等待
	Xlock B	R(A)=12
	B=A-B	A=A*B
	W(B)=6	W(A)=72
	Unlock B	Unlock A

注：R(A)表示读取 A，W(A)表示写入 A

（4）两段锁死锁并发事务调度策略如表 8.3 所示。

表 1.3　两段锁死锁并发事务调度

T_1事务	T_2事务	T_3事务
Slock A R(A)=6		
	Slock A R(A)=6	
Xlock A 等待		
	Xlock A 等待 等待 等待	Slock A R(A)=6 Xlock A 等待

注：R(A)表示读取 A，W(A)表示写入 A

8.9 解释活锁和死锁？简述活锁和死锁的解除方法。

活锁：由于授权封锁的随机性，导致事务 T 的封锁请求一直处于等待状态的封锁。

活锁的解决方法：采用先来先服务的授权策略。

死锁：在并发事务的多个事务进行封锁请求时，分别需要为对方加锁，而对方又不允许加锁，从而造成多方处于一直等待状态。

死锁的解决方法：

（1）预防死锁。

（2）诊断与解除。

第 9 章习题

9.1 简述数据库系统的故障种类。

故障种类：事务故障、系统故障、介质故障和病毒故障等

9.2 简述恢复机制的核心技术以及建立冗余数据的常用技术。

数据恢复技术：建立冗余数据和利用冗余数据实施数据库恢复。

建立冗余数据技术：数据转储和登记日志文件等。

9.3 解释数据转储。简述数据转储的常用方法。举例说明数据转储的可行策略。

数据转储：DBA 定期地把整个数据库复制到磁盘、磁带或光盘等外存。

转储分类：静态海量转储、静态海量转储、动态增量转储和动态增量转储等。

9.4 解释日志文件。简述常用的日志文件及其内容。

日志文件：记录事务对数据库的更新操作的文件。

日志文件分类：记录日志文件和数据块日志文件等。

记录日志文件：以记录为单位的日志文件。内容：

（1）事务标记（事务开始标记，事务结束标记，事务名称等）。

（2）事务操作类型(插入，修改，删除)。

（3）事务操作对象。

（4）更新前数据旧值。

（5）更新后数据新值。

数据块日志文件：以数据块为单位的日志文件。内容：

（1）事务标记（事务开始标记，事务结束标记，事务名称）。

（2）更新前数据块旧值。

（3）更新后数据块新值。

9.5 简述日志文件的登记规则。并解释原因。

（1）严格按照并发事务执行的时间次序依次登记。

（2）先登记日志文件，后更新数据库。

对于规则 2：如果先更新数据库，后登记日志文件，且故障发生在更新数据库之后，登记日志文件之前，则会导致数据库被更新了，而日志文件却没有登记该更新操作，因此在恢复数据库时，将无法从日志文件中得到该更新操作的记录，从而无法恢复该更新操作。

如果先登记日志文件，后更新数据库，且故障发生在登记日志文件之后，更新数据库之前，则日志文件记录了事务对数据库的更新操作，却没有真正更新数据库，因此在恢复数据库时，由于从日志文件中得到了更新数据库操作的记录，经过验证发现，并没有真正更新数据库，所以只需要撤销（UNDO）更新操作，且不会影响数据一致性。

9.6 简述事务故障、系统故障和介质故障的恢复策略。

事务故障恢复方法：利用日志文件撤消事务对数据库的更新操作。

（1）反向扫描文件日志（即：从尾向前扫描），查找事务的更新（INSERT、UPDATE 和 DELETE）操作。

（2）对事务的更新操作执行逆操作。把日志记录中更新前的值写入数据库。

（3）重复（1）和（2），直到读取事务的开始标记。

系统故障恢复方法：撤销未完成事务（UNDO）和重做已完成事务（REDO）。即：

（1）正向扫描日志文件（即：从头向后扫描），建立 REDO 队列和 UNDO 队列。

（2）对 REDO 队列的事务执行重做操作。

（3）对 UNDO 队列的事务执行撤销操作。

介质故障恢复方法：重装数据库，重做已完成事务。

（1）重装数据库。装入距离故障时间最近的最新后备副本，使数据库恢复到最近一次转储时的一致性状态。

（2）重做已完成事务。装入转储结束时刻的日志文件副本，重做已完成的事务。

恢复故障需要 DBA 装入后备副本和日志文件，并由 DBMS 的恢复子系统自动完成。

9.7 解释数据库镜像。

数据（库）镜像：DBMS 把数据库（或其关键数据）复制到备份介质上形成镜像数据，并自动保持镜像数据与主数据的一致。

9.8 解释检查点记录和重新开始文件。

检查点记录：用于记录在检查时间点正在执行的事务清单的日志记录。检查点记录需要登记到日志文件。

重新开始文件：记录检查点记录在日志文件中地址的文件。

9.9 简述检查点机制的恢复策略。

（1）在重新开始文件中，找到最后一个检查点记录在日志文件中的地址，由该地址在日志文件中找到最后一个检查点记录。

（2）由该检查点记录得到检查点建立时刻，所有正在执行的事务清单 ActiveL。

（3）利用 ActiveL 建立重做队列 RedoL 和撤销队列 UndoL 如下：

RedoL ← ∅，即 RedoL 置空。

UndoL ← ActiveL，即：把 ActiveL 放入 UndoL。

（4）从检查点开始正向扫描日志文件，直到日志文件结束。如果有新开始的事务 Ti，则把 Ti 放入 UndoL；如果有提交的事务 Tj，则把 Tj 从 UndoL 移到 RedoL。

（5）对 RedoL 的事务执行 REDO 操作。

（6）对 UndoL 的事务执行 UNDO 操作。

第 10 章习题

10.1　解释下列名词：

（1）数据库设计　　（2）数据字典　　（3）游标

（4）ODBC　　（5）JDBC　　（6）物理结构

数据库（系统）设计：针对实际应用，设计规范优化的逻辑模型和物理结构，进而建立相应的数据库及其应用系统，从而科学有效地管理数据，满足用户的应用需求。

数据字典：数据库中数据的详细描述。

游标：指向数据缓冲区 SQLCA 的指针。

ODBC：Open Database Connectivity；开放数据库互连。数据库引擎数国际标准接口。

JDBC：Java DataBase Connectivity；Java 数据库连接。Java 应用程序连接数据库的标准接口和方法。

物理结构（内模式）：数据库在存储介质上的存储结构和存放位置。

10.2　简述数据库设计的方法和步骤。

数据库设计方法：手工经验法、新奥尔良法、E-R 图法、3NF 法和面向对象的 DB 设计法和统一建模语言 UML 等。

数据库设计步骤：需求分析、概念结构设计、逻辑结构设计、物理结构设计、系统实施和运行与维护等。

10.3　简述需求分析的方法和步骤。

需求分析方法：跟班、座谈、专人介绍、询问、调查表和工作日志。

需求分析步骤：

（1）调查组织机构。

（2）调查部门业务活动。

（3）明确用户和系统的要求。

（4）分析表达用户需求，确定系统边界。

（5）建立数据字典，撰写需求分析文档。

10.4　简述数据字典的内容。

数据项、数据结构、数据流、数据存储和处理过程等。

10.5　简述概念结构设计的方法和步骤。

概念结构设计方法：自顶向下、自底向上、逐步扩张和混合策略等。

概念结构设计步骤：概念模型的抽象、局部 E-R 图设计、全局 E-R 图设计和概念结构的优化等。

10.6　简述全局 E-R 图的集成方法及其冲突。

集成方法：整体集成法和逐步集成法等。

集成冲突：属性冲突、命名冲突和结构冲突等。

10.7　简述逻辑结构设计的内容。

（1）概念模型向逻辑模型的转换。E-R 图向关系模式的转换。

（2）逻辑模型完整性设计。关系模式的完整性设计。

（3）逻辑模型规范化。关系模式的规范化和优化。

（4）外模式设计。面向用户的子模式设计及其视图实现。

10.8　简述物理结构设计的内容。

存取方法、存储结构、存放位置和存储介质等。

10.9　解释聚簇索引，简述聚簇索引与非聚簇索引的区别。

聚簇索引：在按照关键属性建立索引时，需要按照索引顺序对相应元组的物理存储位置进行重新组织，使得索引顺序与相应元组的物理顺序始终保持一致。

聚簇索引与非聚簇索引区别：

（1）前者的顺序与数据的物理存储顺序一致；后者的顺序与数据物理顺序无关。

（2）前者 B+树的叶节点就是数据节点；后者 B+树的叶节点是索引节点，其指针指向对应的元组（数据块）。

（3）一个关系只能有一个聚簇索引；非聚簇索引则可以有多个。

（4）维护后者的开销较小，而维护前者的开销则较大。

（5）前者适合于更新较少的应用，而后者则适合于更新较多的应用。

（6）前者灵活性较差，后者则相对比较灵活。

10.10　对于嵌入式 SQL，如何区分 SQL 语句和主语言语句？

在主语言中，使用专用的关键字或者专用类的专用方法直接调用 SQL 语句。

例如：在 VC++中，使用

10.11　解释系统评价。简述系统评价的内容。

系统评价：通过对系统运行过程和绩效的审查，检查系统功能和性能是否达到预期的目标，是否充分利用了系统的各种资源，管理工作是否完善，并且给出系统调整和改进的建议。

内容：目标与功能评价、性能评价、经济效益评价和撰写评价报告等。

10.12　简述数据库维护的内容。

（1）数据库的转储和恢复。

（2）数据库的安全性控制。

（3）数据库的完整性控制。

（4）数据库性能维护。

（5）数据库的重组与重构。

第 11 章习题

11.1　解释 XML，简述 XML 数据库的两种基本类型及其区别。

XML：用于网络数据交换，并且可以自行定义标记的可扩展描述型标记语言（网络数据交换标准）。

XML 两种类型：NXD 和 XED。区别：

NXD 是专门对 XML 数据格式的文档进行存取管理和数据查询的数据库技术。关键技术是数据存储、查询处理、事务处理、代数系统和模式规范化。

XED 是在传统数据库的基础上，通过增加对 XML 数据的映射功能，从而实现对 XML 数据进行存取管理的数据库技术。关键技术是 XML 文档与关系模式的映射与存储。

NXD 和 XED 的优点和缺点不同。

11.2　解释数据仓库，简述数据仓库的基本特征、数据模型和设计过程。

数据仓库：面向主题的、集成的、稳定的、时变的数据集合，用以支持经营管理中的决策制定过程。

基本特征：面向主题、集成的、稳定的、时变的和集合性等。

数据模型：多维数据模型（星型模型、星系模型、星座模型和雪花模型）。

设计过程：规划与分析、设计与实施（概念模型设计、逻辑模型设计、物理模型设计、源数据的 ETL 设计、数据表达与访问设计和维护方案设计；重点）、数据仓库应用等。

11.3　简述数据库系统和数据仓库系统的区别与联系。

区别：

（1）主题性。DBS 是面向事务，提供处理结果；DWS 是面向主题，提供决策信息。

（2）对象性。DBS 是面向操作人员；DWS 是面向决策人员。

（3）集成性。DBS 是通过统计计算，进行 OLTP；DWS 是通过 ETL,进行 OLAP。

（4）更新性。DBS 数据可以更新；DWS 数据不可更新。

（5）时变性。DBS 数据可以随时改变；DWS 只能增加新数据，不能修改数据。

联系：

DBS 是 ROLAP 型 DWS 的基础；DBS 和 DWS 的设计过程雷同；DWS 的基础数据来自 DBS；DWS 的数据仓库是 DBS 的数据库经过行 ETL 的结果。

11.4　解释 ETL，简述 ETL 的过程。

ETL：把 OLTP 系统中的数据抽取出来，并把不同数据源的数据进行转换、清洗、整合和规范化，然后装入数据仓库的过程。。

过程：数据抽取、数据转换、数据清洗和数据加载等。

11.5　解释 OLTP，简述研发 OLTP 系统应该遵守的基本准则。

OLTP：联机事务处理。利用计算机网络，将分布在不同地理位置的业务处理计算机设备或

者网络与业务管理网络中心连接，以便在任何网络节点进行统一、实时的业务处理活动或者客户服务。

基本准则：OLTP 与决策工作量、数据放置与文件组、调整 OLTP 事务、控制数据内容、数据备份、索引等。

11.6　解释 OLAP 及其基本操作，简述 OLTP 与 OLAP 的区别。

OLAP：联机分析处理。使分析人员能够迅速、一致、交互地从各个方面观察信息，以达到深入理解数据的目的软件技术。

基本分析操作：多维数据的切片、切块、旋转，钻取等。

OLTP 和 OLAP 的区别：背景和目的、数据模型、综合程度、更新和处理方式等存在差异。即 OLAP 关注如何理解大量数据。目的是通过分析数据，为决策人员和高层管理人员提供决策信息。具体特征包括导出、综合、历史数据；面向决策人员，支持管理需要；面向分析，分析驱动；不可更新，周期性刷新；数据处理量大。OLTP 关注如何处理大量数据。目的是面向操作人员和低层 DBA 完成常规事物处理。具体特征包括原始、细节、当前数据；面向操作人员，支持日常操作；面向应用，事务驱动；可更新；数据处理量小。

11.7　解释数据挖掘，简述数据挖掘模型。

数据挖掘：从大量的、不完全的、有噪声的、模糊的、随机的实际数据中，提取隐含其中的、未知的、潜在的、有用的信息和知识的过程。

数据挖掘模型：CRISP/DM 模型。CRoss Industry Standard Process for Data Mining。

（1）业务理解（Business Understanding）。

（2）数据理解（Data Understanding）。

（3）数据准备（Data Preparation）。

（4）建立模型（Modeling）。

（5）模型评估（Evaluation）。

（6）部署（Deployment）。

11.8　解释关联规则，简述关联规则算法的基本原理。

关联规则：从数据库中发现属性之间未知的、隐藏的关系，从而从大量交易记录中发现有意义的关联规则。

算法：Apriori 算法、AIS 算法、SETM 算法、DHP 算法、PARTITION 算法、Sampling 算法和 FP Growth 算法等。

Apriori 算法：商品集 $W=\{w_1,w_2,...,w_u\}$，交易集 $E=\{e_1,e_2,...,e_v\}$，交易子集 $A=\{a_1,a_2,...,a_m\}$、$B=\{b_1,b_2,...,b_n\}$ 和 $A\cap B=\{a_1,a_2,...,a_m,b_1,b_2,...,b_n\}$（既购买商品 A 又购买商品 B）。$e_j\in W(j=1,2,...,v)$；$a_k,b_l\in E(k=1,2,m;l=1,2,...,n)$。并且交易集和交易子集的记录个数分别为 $|E|=e$；$|A|=x$；$|B|=y$；$|A\cap B|=z$。则对于关联规则 $A\to B$，定义其可信度和支持度：

可信度：$C(A \to B) = \frac{z}{x}$；支持度：$S(A \to B; E) = \frac{z}{e}$。

结论：通过支持度和可信度与阈值的关系，确定 A 和 B 之间的相关性。

11.9　解释聚类，简述常用聚类算法及其基本原理。

聚类：把数据集划分为若干类的过程，并使同类的数据对象具有较高的相似度；而不同类的数据对象则相反。

算法：划分聚类（K 均值、C 均值和 K 中心等）、层次聚类（Cure、Chameleon 和 Birch 等）、密度聚类（DBScan、Optics 和 DenClue 等）、网格聚类（STING、ClIQue 和 Wave Cluster）和模型聚类（Cobweb 和神经网络算法）等。

2）K 均值：已知聚类个数时，实现分类的无监督聚类算法。首先随机选择 k 个数据点作为聚类中心；然后计算其它点到聚类中心的距离，通过对距离均值的计算，不断改变聚类中心的位置，直到聚类中心稳定。优点：算法简单，收敛速度较快；缺点：需要事先知道聚类个数；而且要求类内高聚合，类间低耦合；均值计算必须定义适当等。

3）K 中心：K 均值聚类的改进。首先为每类随机选择中心样本点，并把剩余样本点按照距离远近分配给最近的类；然后用非中心点替换中心点，并检查聚类代价，若聚类代价小于零，则执行替换，直到中心点不再发生变化。

4）C 均值：选取类数 k，并从样本集中任意取定 k 个向量 $c_1, c_2, ..., c_k$ 作为聚类中心，则第 1 步，把每个样本 $x_l = [x_{l1}, ..., x_{ln}]$，按照下式归入中心为 c_i 的类 C_i；

$$\|x_l - c_i\| = \min_j \|x_l - c_j\|$$

第 2 步，按照下式重新调整聚类中心 $c_i = [c_{i1}, \ldots, c_{in}]^T$；第 3 步，若第 2 步聚类中心不再变化，则终止。否则转至第 2 步。

$$c_{im} = (\sum x_{l_j m}) / N_i \text{（} N_i \text{是} C_i \text{的向量数）}$$

11.10　解释决策树，简述决策树算法的基本原理。

决策树：如果训练样本的所有样本是同类的，则把它作为叶节点，节点内容是类别标记；否则，根据预定策略选择一个属性，并且按照属性值，将样本划分为若干子集，使得每个子集上样本的属性值相同；然后依次递归处理每个子集。

算法是 Quiulan 的 ID3（Iterative Dichotomiser 3）算法，然后推广到 C4.5、CART 和 Assistant 算法。

ID3 的思想：

（1）任意选取一个属性作为决策树的根结点，然后按照该属性的值，创建树的分支。

（2）如果叶结点的实例属于同类，则用类标记标识叶结点；如果所有叶结点都有类标记，则算法终止；否则，选取一个从该结点到根节点的路径中没有出现过的属性为标记标识该结点，然后使用这个属性所有的取值继续创建树的分支；并重复该步。

11.11　解释分类，简述常用分类算法及其基本原理。

分类：通过分析训练样本数据，产生关于类别（分类规则）的精确描述。目的是通过创建分类模型，把数据映射到给定的类别。

算法：决策树、KNN 分类（K 最近邻法）、SVM 分类（支持向量机）、VSM 分类（向量空间模型）、Autoclass 分类（Bayesian 网络，无监督）、神经网络分类（反向传播 BP 网络）、示例推理分类器、遗传算法分类（选择、交叉、变异）、粗糙分类（等价关系、上下近似集）和模糊分类（隶属函数、截集）等。

11.12　解释预测，简述常用预测算法及其基本原理。

预测：首先构造模型，然后使用模型预测未知值。

算法：回归分析（线性回归、多元回归和非线性回归等）、趋势外推和时间序列等。

线性回归：利用直线 $y=\alpha+\beta x$ 描述数据模型。已知样本 (x_1,y_1)，…，(x_n,y_n)，则利用最小二乘法计算的回归系数 α 和 β 如下：

$$\beta=\frac{\sum_{i=1}^{n}(x_i-\bar{x})(y_i-\bar{y})}{\sum_{i=1}^{n}(x_i-\bar{x})^2};\ \alpha=\bar{y}-\beta\bar{x};\ \bar{x}=\frac{1}{n}\sum_{i=1}^{n}x_i;\ \bar{y}=\frac{1}{n}\sum_{i=1}^{n}y_i$$

多元回归：由多维变量组成的线性回归函数 $y=\alpha+\beta_1x_1+\beta_2x_2$；利用最小二乘法计算回归参数 α，β_1 和 β_2。经典算法是 Polynom 算法，即对于预测变量 y 和影响因素 x_1，…，x_n。若满足 $y=a_1x_1+a_2x_2+\cdots+a_nx_n$，则根据已知数据算出系数 a_1，…，a_n。

11.13　简述数据仓库与数据挖掘的应用。

银行、金融、电信、保险、医药、交通、税务、零售等成功应用；Web 数据、生物信息、文本、音频、视频和空间信息等逐步应用。

11.14　解释大数据，简述大数据的特征。

大数据：无法在一定时间范围内，用常规软件工具进行捕捉、管理和处理的数据集合。

特征：Volume（巨量）、Velocity（高速）、Variety（多样）、Value（价值）和 Veracity（真实）等。

11.15　简述大数据系统及其应用。

（1）企业大数据分析与预测。

（2）政府大数据分析与预测。

（3）服务大数据分析与预测。

（4）消费/零售大数据分析与预测。

（5）工业/农业大数据分析与预测。

（6）通信/交通大数据分析与预测等。

实验 1　建立数据库

（1）SQL Server 2016 的安装、配置、启动、基本操作和退出方法。

（2）使用对象资源管理器，在默认位置，使用默认参数建立数据库 JimInfo。

（3）使用对象资源管理器，在 D 盘的 MyData 文件夹中，建立数据库 EBook。

要求：EBook 的逻辑名称为 EBook，初始大小为 10MB，增量为 10MB，不限制增长，物理文件名称为 EBook.mdf。EBook 的日志的逻辑名称为 EBookLog，初始大小为 5MB，增量为 5%，增长的最大值限制为 2000MB，物理文件名称为 EBookLog.ldf。

（4）使用查询编辑器，在默认位置，使用默认参数建立数据库 MimInfo。

（5）使用查询编辑器，在 D 盘的 MyData 文件夹中，建立数据库 CInfo。

要求：CInfo 的逻辑名称为 CInfo，初始大小为 10MB，增量为 10MB，不限制增长，物理文件名称为 ComInfo.mdf。CInfo 的日志的逻辑名称为 CInfoLog，初始大小为 5MB，增量为 5%，增长的最大值限制 2000MB，物理文件名称为 ComInfo.ldf。

（6）使用对象资源管理器，删除数据库 JimInfo。使用查询编辑器，删除数据库 MimInfo。

（1）课外自行完成。

（2）、（3）和（6）使用对象资源管理器自行完成。

（4）使用查询编辑器的参考代码：

```
-- “--”开头的为注释语句
-- 建立默认数据库 MimInfo
CREATE DATABASE MimInfo
```

（5）使用查询编辑器的参考代码：

```
-- 建立指定数据库 CInfo，首先需要建立目录 D:\MyData
  CREATE DATABASE CInfo
    ON (NAME='CInfo',FILENAME='D:\MyData\ComInfo.mdf',
              SIZE=10MB,FILEGROWTH=10MB,MAXSIZE=UNLIMITED)
    LOG ON (NAME='CInfoLog',FILENAME='D:\MyData\ComInfo.ldf',
              SIZE=5MB,FILEGROWTH=5%,MAXSIZE=2000MB)
```

（6）使用查询编辑器的参考代码：

```
-- 打开系统数据库 Master
 USE Master
-- 删除数据库 MimInfo
 DROP DATABASE MimInfo
```

实验 2　数据完整性

已知商品销售数据库 CInfo 及其职工表、商品表、商店表和销售表如下：

职工 Emp（工号 ENo，姓名 EName，性别 ESex，生日 Birth，电话 Phone，店号 SNo，工资 Salary，聘期 Term，职补 Subs，经理 Head）

商品 Ware（品号 WNo，品名 WName，厂商 Maker，厂址 MAddr，单价 Price，数量 Quant）

商店 Shop（店号 SNo，店名 SName，店址 SAddr，电话 Phone）

销售 Sale（店号 SNo，品号 WNo，销量 SVol）

请在“实验 1”创建的 CInfo 中，创建上述关系，设计并实现相应的完整性约束。

```
--打开数库：使用数据库之前，必须先打开数据库。
USE CInfo
--建立商店表
CREATE TABLE Shop(
    SNo CHAR(6) PRIMARY KEY
        CHECK (SNo LIKE 'S[0-9][0-9][0-9][0-9][0-9]'),
    SName CHAR(22) NOT NULL,
    SAddr CHAR(36) NOT NULL,
    Phone CHAR(12) UNIQUE CHECK (Phone LIKE
        '[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'))
--建立职工表
CREATE TABLE Emp(
    ENo CHAR(6) PRIMARY KEY
        CHECK (ENo LIKE '[A-Z][0-9][0-9][0-9][0-9][0-9]'),
    EName CHAR(8) NOT NULL,
    ESex CHAR(2) NOT NULL CHECK (ESex='男' OR ESex='女'),
    Birth DATE,
    Phone CHAR(12) NOT NULL CHECK (Phone LIKE
        '[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'),
    SNo CHAR(6) REFERENCES Shop(SNo),
    Salary REAL,
    Term DECIMAL(3,1) DEFAULT 5,
    Subs FLOAT,
    Head CHAR(6) REFERENCES Emp(ENo))
```

```
--建立商品表
CREATE TABLE Ware(
   WNo CHAR(6) PRIMARY KEY
      CHECK (WNo LIKE 'W[0-9][0-9][0-9][0-9][0-9]'),
   WName CHAR(22) NOT NULL,
   Maker CHAR(22) NOT NULL,
   MAddr CHAR(36) NOT NULL,
   Price DECIMAL(9,2) CHECK (Price>0 AND Price<=999),
   Quant INT CHECK (Quant>=0))
--建立销售表
CREATE TABLE Sale(
   SNo CHAR(6),
   WNo CHAR(6),
   SVol INT,
   PRIMARY KEY (SNo,WNo),
   FOREIGN KEY (SNo) REFERENCES Shop(SNo)
      ON DELETE CASCADE ON UPDATE CASCADE,
   FOREIGN KEY (WNo) REFERENCES Ware(WNo)
      ON DELETE CASCADE ON UPDATE CASCADE)
```

实验 3　学籍数据库及其完整性

已知学籍数据库 SInfo 及其学生表、课程表、选课表、专业表、教师表和讲授表如下：

学生 S（学号 SNo，姓名 SName，性别 SSex，生日 Birth，专业号 MNo）

课程 C（课程号 CNo，课程名 CName，先修课 CPNo，学分 Credit）

选课 SC（学号 SNo，课程号 CNo，成绩 Grade）

专业 M（专业号 MNo，专业名 MName，学院 Coll）

教师 T（工号 TNo，姓名 TName，性别 TSex，职称 Title，婚否 Marry）

讲授 TC（工号 TNo，课程号 CNo，教学效果 TEffe）

请完成如下任务：

（1）创建学籍数据库 SInfo。

（2）创建上述关系及其相应的完整性约束

```
-- 需要首先在 D 盘建立 MyData 文件夹
CREATE DATABASE SInfo
    ON (NAME='SInfo',FILENAME='D:\MyData\SInfo.mdf')
    LOG ON (NAME='SInfoLog',FILENAME='D:\MyData\SInfo.ldf ')
GO
USE SInfo
CREATE TABLE M(
    MNo CHAR(5) CONSTRAINT MNo_PK PRIMARY KEY NOT NULL,
    MName CHAR(20) NOT NULL,
    Coll CHAR(22))
CREATE TABLE S(
    SNo CHAR(10) CONSTRAINT SNo_PK PRIMARY KEY NOT NULL,
    SName CHAR(8) NOT NULL,
    SSex CHAR(2) CHECK (SSex='男' OR SSex='女'),
    Birth DATE,
    MNo CHAR(5) CONSTRAINT MNo_FK REFERENCES M(MNo))
CREATE TABLE C(
    CNo CHAR(6) CONSTRAINT CNo_PK PRIMARY KEY NOT NULL,
    CName CHAR(40) NOT NULL,
    CPNo CHAR(6),
    Credit INT,
    FOREIGN KEY (CPNo) REFERENCES C(CNo))
```

```
CREATE TABLE SC(
    SNo CHAR(10) NOT NULL,
    CNo CHAR(6) NOT NULL,
    Grade FLOAT CHECK (Grade>=0 AND Grade<=100),
    CONSTRAINT SNo_CNo_PK PRIMARY KEY (SNo, CNo),
    FOREIGN KEY (SNo) REFERENCES S(SNo)
      ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (CNo) REFERENCES C(CNo)
      ON DELETE CASCADE ON UPDATE CASCADE)
CREATE TABLE T(
    TNo CHAR(6) PRIMARY KEY NOT NULL,
    TName CHAR(8) NOT NULL,
    TSex CHAR(2) CHECK (TSex='男' OR TSex='女'),
    Title CHAR(6) CHECK (Title IN ('助教','讲师','副教授','教授')),
    Marry CHAR(2) CHECK (Marry='是' OR Marry='否'))
CREATE TABLE TC(
    TNo CHAR(6) NOT NULL,
    CNo CHAR(6) NOT NULL,
    TEffe CHAR(6) CHECK (TEffe IN ('优秀','良好','一般','较差')),
    PRIMARY KEY (TNo, CNo),
    FOREIGN KEY (TNo) REFERENCES T(TNo)
      ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (CNo) REFERENCES C(CNo)
      ON DELETE CASCADE ON UPDATE CASCADE)
```

实验 4　编辑完善表

（1）在“实验 1”创建的 CInfo 中，编辑和完善职工 Emp、商品 Ware、商店 Shop 和销售 Sale 等表的结构。

（2）根据 Emp、Ware、Shop 和 Sale 的结构和完整性约束，分别编辑相应的记录，具体内容自定。

```
-- 建立指定数据库 CInfo，首先需要建立目录 D:\MyData
CREATE DATABASE CInfo
  ON (NAME='CInfo',FILENAME='D:\MyData\ComInfo.mdf',
            SIZE=10MB,FILEGROWTH=10MB,MAXSIZE=UNLIMITED)
  LOG ON (NAME='CInfoLog',FILENAME='D:\MyData\ComInfo.ldf',
            SIZE=5MB,FILEGROWTH=5%,MAXSIZE=2000MB)
GO
--打开数库：使用数据库之前，必须先打开数据库。
USE CInfo
--建立商店表
CREATE TABLE Shop(
  SNo CHAR(6) PRIMARY KEY
    CHECK (SNo LIKE 'S[0-9][0-9][0-9][0-9][0-9]'),
  SName CHAR(22) NOT NULL,
  SAddr CHAR(36) NOT NULL,
  Phone CHAR(12) UNIQUE CHECK (Phone LIKE
    '[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'))
--建立职工表
CREATE TABLE Emp(
  ENo CHAR(6) PRIMARY KEY
    CHECK (ENo LIKE '[A-Z][0-9][0-9][0-9][0-9][0-9]'),
  EName CHAR(8) NOT NULL,
  ESex CHAR(2) NOT NULL CHECK (ESex='男' OR ESex='女'),
  Birth DATE,
  Phone CHAR(12) NOT NULL CHECK (Phone LIKE
    '[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'),
  SNo CHAR(6) REFERENCES Shop(SNo),
  Salary REAL,
```

```
   Term DECIMAL(3,1) DEFAULT 5,
   Subs FLOAT,
   Head CHAR(6) REFERENCES Emp(ENo))
--建立商品表
CREATE TABLE Ware(
   WNo CHAR(6) PRIMARY KEY
      CHECK (WNo LIKE 'W[0-9][0-9][0-9][0-9][0-9]'),
   WName CHAR(22) NOT NULL,
   Maker CHAR(22) NOT NULL,
   MAddr CHAR(36) NOT NULL,
   Price DECIMAL(9,2) CHECK (Price>0 AND Price<=9999),
   Quant INT CHECK (Quant>=0))
--建立销售表
CREATE TABLE Sale(
   SNo CHAR(6),
   WNo CHAR(6),
   SVol INT,
   PRIMARY KEY (SNo,WNo),
   FOREIGN KEY (SNo) REFERENCES Shop(SNo)
      ON DELETE CASCADE ON UPDATE CASCADE,
   FOREIGN KEY (WNo) REFERENCES Ware(WNo)
      ON DELETE CASCADE ON UPDATE CASCADE)
--给商店添加纪录
INSERT INTO Shop VALUES(
   'S00001','华润超市文一店','杭州市文一路 66 号','013666666666')
INSERT INTO Shop VALUES(
   'S00002','华润超市文二店','杭州市文二路 66 号','013999999999')
INSERT INTO Shop VALUES(
   'S00003','华润超市文三店','杭州市文三路 66 号','013666666699')
--给商品添加纪录
INSERT INTO Ware VALUES(
   'W00001','SONY 电视机','SONY 公司','杭州市下沙学正街号',9900,66)
INSERT INTO Ware VALUES(
   'W00002','DELL 计算机','DELL 公司','杭州市下沙学正街号',8800,99)
```

```
--给职工添加纪录
INSERT INTO Emp VALUES(
   'A00001','张三','男','1996-06-09','013666699999','S00001',5000,6,2200,'A00001')
INSERT INTO Emp VALUES(
   'A00002','李四','女','1998-09-19','013999966666','S00001',3000,5,0.00,'A00001')
INSERT INTO Emp VALUES(
   'A00003','王五','男','2000-11-16','013669696969','S00002',6000,9,3000,'A00003')
INSERT INTO Emp VALUES(
   'A00004','孙六','女','1992-10-10','013996969696','S00002',4000,8,0.00,'A00003')
INSERT INTO Emp VALUES(
   'A00005','马七','女','1990-12-12','013996969696','S00002',4000,9,0.00,'A00003')
--给销售添加纪录
INSERT INTO Sale VALUES('S00001','W00001',10)
INSERT INTO Sale VALUES('S00001','W00002',20)
INSERT INTO Sale VALUES('S00002','W00001',16)
INSERT INTO Sale VALUES('S00002','W00002',26)
INSERT INTO Sale VALUES('S00003','W00001',11)
```

实验 5　数据查询

在“实验 4”的 CInfo 及其 Emp、Ware、Shop 和 Sale 中，完成如下查询：

（1）在销售中，查询销量大于 16 的销售信息。

（2）在销售中，查询销量小于等于 16 的店号和品号。

（3）在销售中，统计每类商品的品号和销量。

（4）在销售中，统计至少销售两次商品的店号和销量。

（5）在销售中，查询全部销售信息，并按照销量从大到小排序。

（6）在销售中，查询售量最高的店号和品号。

（7）在销售中，查询每类售出商品的品号以及销量的最高、最低和平均值。

（8）在销售中，查询售出每类商品的品号和销售额。

（9）在销售中，查询售出每类商品的品号。

（10）在销售中，查询售出商品的类数。

（11）在销售中，查询销售品号为 W00001 的商店的个数。

（12）查询职工,商店,商品,销售的笛卡尔积。

（13）查询在店号和品号后 5 位相等时的店号,店名,品号,品名（条件连接）。

（14）查询每个商店销售商品的详细信息（等值连接）。

（15）查询每个商店销售商品的详细信息（自然连接）。

（16）查询李四所在的商店，显示字段为姓名、店名、店址、商店的电话。

（17）查询王五所在商店的销量，显示字段为工号、姓名、店号、销量。

（18）查询王五所在的商店的销量，显示字段为姓名、店名、销量。

（19）查询华润超市文二店的销量，显示字段为店名、销量。

（20）查询 SONY 电视机的销量，显示字段为品名、销量。

（21）查询华润超市文一店销售 SONY 电视机的销量，显示字段为店名、品名、销量。

（22）王五所在商店销售 SONY 电视机的销量，显示字段为姓名、店名、品名、销量。

（23）工号为 A00001 的职工所在商店销售品号为 W00002 的商品的店名、品名和销量。

（24）查询工号为 A00002 的职工所在商店的经理的姓名。

（25）查询王五经理所在商店的总售量,并输出经理姓名和总销量。

（26）查询所有职工所在的所有商店的所有销售商品的详细信息。

```
SELECT *    FROM Sale WHERE SVol>16
SELECT SNo, WNo FROM Sale WHERE SVol<=16
SELECT WNo, SUM(SVol) SVol FROM Sale GROUP BY WNo
SELECT SNo, SUM(SVol) FROM Sale GROUP BY SNo HAVING COUNT(*)>=2
SELECT * FROM Sale ORDER BY SVol DESC
```

```
SELECT SNo, WNo, SVol FROM Sale WHERE SVol=
   (SELECT MAX(SVol) FROM Sale)
SELECT WNo,MAX(SVol) 最高,MIN(SVol) 最低,AVG(SVol) 平均
   FROM Sale GROUP BY WNo
SELECT Sale.WNo, SUM(SVol*Price) 销售额 FROM Sale, Ware
   WHERE Ware.WNo=Sale.WNo GROUP BY Sale.WNo
SELECT DISTINCT WNo FROM Sale
SELECT WNo, COUNT(*) FROM Sale GROUP BY WNo
SELECT COUNT(DISTINCT SNo) FROM Sale WHERE WNo='W00001'
SELECT * FROM Emp,Shop,Ware,Sale
SELECT SNo,SName,WNo,WName FROM Shop,Ware
   WHERE RIGHT(SNo,5)=RIGHT(WNo,5)
SELECT * FROM Shop,Ware,Sale
   WHERE Shop.SNo=Sale.SNo AND Ware.WNo=Sale.WNo
SELECT Shop.SNo,SName,SAddr,Phone,
      Ware.WNo,WName,Maker,MAddr,Price,Quant, SVol
   FROM Shop,Ware,Sale
   WHERE Shop.SNo=Sale.SNo AND Ware.WNo=Sale.WNo
SELECT EName,SName,SAddr,Shop.Phone FROM Emp,Shop
   WHERE EName='李四' AND Emp.SNo=Shop.SNo
SELECT ENo,EName,Sale.SNo, SVol FROM Emp,Sale
   WHERE EName='王五' AND Emp.SNo=Sale.SNo
SELECT EName,SName, SVol FROM Emp,Shop,Sale
   WHERE EName='王五' AND Emp.SNo=Shop.SNo AND Emp.SNo=Sale.SNo
SELECT SName, SVol FROM Shop,Sale
   WHERE SName='华润超市文二店' AND Shop.SNo=Sale.SNo
SELECT WName, SVol FROM Ware,Sale
   WHERE WName='SONY 电视机' AND Ware.WNo=Sale.WNo
SELECT SName,WName, SVol FROM Shop,Ware,Sale
   WHERE SName='华润超市文一店' AND WName='SONY 电视机' AND
            Shop.SNo=Sale.SNo AND Ware.WNo=Sale.WNo
SELECT EName,SName,WName,SVol FROM Emp,Shop,Ware,Sale
   WHERE EName='王五' AND WName='SONY 电视机' AND
            Shop.SNo=Sale.SNo AND
```

```
            Ware.WNo=Sale.WNo AND
            Emp.SNo=Sale.SNo
SELECT Shop.SName,WName, SVol FROM Shop,Ware,Sale,Emp
   WHERE Shop.SNo=Sale.SNo AND Ware.WNo=Sale.WNo AND
            Shop.SNo=Emp.SNo AND ENo='A00001' AND Ware.WNo='W00002'
SELECT X.EName FROM Emp X,Emp Y
   WHERE Y.ENo='A00002' AND Y.Head=X.ENo
SELECT '王五' 经理, SUM(SVol) 总销量 FROM Sale WHERE SNo=
   (SELECT SNo FROM Emp
       WHERE EName='王五' AND ENo=Head)
SELECT Emp.*,Shop.*,Ware.*,Sale.*
FROM Emp,Shop,Ware,Sale
WHERE Shop.SNo=Sale.SNo AND
         Ware.WNo=Sale.WNo AND
         Emp.SNo=Sale.SNo
```

实验 6　学籍数据库及其数据查询

在“实验 1”创建的 SInfo 中，编辑和完善学生 S、课程 C、选课 SC、专业 M、教师 T 和讲授 TC 等表的结构和完整性约束，并分别编辑相应的记录（具体内容自定）。

在 SInfo 中，完成如下查询：

（1）查询女生的基本信息。

（2）查询 1997 之前（小于）或者 1999 之后（大于）出生的学生的姓名、生日和学院。

（3）查询信息学院和外语学院的学生姓名。

（4）查询课程号为 020101 的课程信息。

（5）查询姓马的学生的姓名、学号和性别。

（6）查询姓“刘”且第三个汉字为“鑫”的学生姓名。

（7）查询以“Java_”开头，且倒数第 2 个字符为 y 的课程的详细情况。

（8）查询全部选课信息，查询结果先按课程号升序排序，然后再按学号降序排序。

（9）计算课程号为 020101 的课程的课程号、最高、最低和平均成绩。

（10）统计各门课程及其选修人数。

（11）查询选修 6 门以上课程的学生学号。

（12）查询至少 6 门课程是 90 分以上的学生学号。

（13）查询每一门课的先修课的先修课。

（14）查询每个学生的学号、姓名、选修的课程名及成绩。

（15）查询与李光同专业的学生的学号和姓名。

（16）查询选修“图像分析”的学生姓名。

（17）查询比 M0002 专业某个学生生日小的非 M0002 专业的学生姓名。

（18）查询比 M0003 专业所有学生生日都小的非 M0003 专业的学生姓名。

（19）查询选修全部课程的学生姓名。即没有一门课是他不选的。

（20）查询至少选修学号为 2005010101 的学生选修的全部课程的学生学号。

```
-- 需要首先在 D 盘建立 MyData 文件夹
CREATE DATABASE SInfo
    ON (NAME='SInfo',FILENAME='D:\MyData\SInfo.mdf')
    LOG ON (NAME='SInfoLog',FILENAME='D:\MyData\SInfo.ldf ')
GO
USE SInfo
CREATE TABLE M(
    MNo CHAR(5) CONSTRAINT MNo_PK PRIMARY KEY NOT NULL,
    MName CHAR(20) NOT NULL,
```

```
    Coll CHAR(22))
CREATE TABLE S(
    SNo CHAR(10) CONSTRAINT SNo_PK PRIMARY KEY NOT NULL,
    SName CHAR(8) NOT NULL,
    SSex CHAR(2) CHECK (SSex='男' OR SSex='女'),
    Birth DATE,
    MNo CHAR(5) CONSTRAINT MNo_FK REFERENCES M(MNo))
CREATE TABLE C(
    CNo CHAR(6) CONSTRAINT CNo_PK PRIMARY KEY NOT NULL,
    CName CHAR(40) NOT NULL,
    CPNo CHAR(6),
    Credit INT,
    FOREIGN KEY (CPNo) REFERENCES C(CNo))
CREATE TABLE SC(
    SNo CHAR(10) NOT NULL,
    CNo CHAR(6) NOT NULL,
    Grade FLOAT CHECK (Grade>=0 AND Grade<=100),
    CONSTRAINT SNo_CNo_PK PRIMARY KEY (SNo, CNo),
    FOREIGN KEY (SNo) REFERENCES S(SNo)
      ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (CNo) REFERENCES C(CNo)
      ON DELETE CASCADE ON UPDATE CASCADE)
CREATE TABLE T(
    TNo CHAR(6) PRIMARY KEY NOT NULL,
    TName CHAR(8) NOT NULL,
    TSex CHAR(2) CHECK (TSex='男' OR TSex='女'),
    Title CHAR(6) CHECK (Title IN ('助教','讲师','副教授','教授')),
    Marry CHAR(2) CHECK (Marry='是' OR Marry='否'))
CREATE TABLE TC(
    TNo CHAR(6) NOT NULL,
    CNo CHAR(6) NOT NULL,
    TEffe CHAR(6) CHECK (TEffe IN ('优秀','良好','一般','较差')),
    PRIMARY KEY (TNo, CNo),
    FOREIGN KEY (TNo) REFERENCES T(TNo)
```

```
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (CNo) REFERENCES C(CNo)
        ON DELETE CASCADE ON UPDATE CASCADE)
GO
INSERT INTO M VALUES ('M0001','数学','数学学院')
INSERT INTO M VALUES ('M0002','英语','外语学院')
INSERT INTO M VALUES ('M0003','软件工程','信息学院')
INSERT INTO S VALUES ('2005010101','张明','男','1996-11-16','M0001')
INSERT INTO S VALUES ('2005010102','李光','女','1997-10-19','M0001')
INSERT INTO S VALUES ('2005020103','王武','男','2002-09-12','M0002')
INSERT INTO S VALUES ('2005020104','孙文','女','1998-06-10','M0002')
INSERT INTO S VALUES ('2005030105','马松','女','1999-12-11','M0003')
INSERT INTO S VALUES ('2005030106','刘金','男','1997-06-26','M0003')
INSERT INTO C VALUES ('020101','英语',NULL,4)
INSERT INTO C VALUES ('010101','高等数学','020101',6)
INSERT INTO C VALUES ('030101','数据结构','020101',3)
INSERT INTO C VALUES ('030102','图像分析','010101',3)
INSERT INTO C VALUES ('030201','软件工程','030101',2)
INSERT INTO C VALUES ('030202','数据库原理','030101',2)
INSERT INTO SC VALUES ('2005010101','010101',99)
INSERT INTO SC VALUES ('2005010102','030101',95)
INSERT INTO SC VALUES ('2005020103','020101',93)
INSERT INTO SC VALUES ('2005020104','030102',96)
INSERT INTO SC VALUES ('2005030105','030201',97)
INSERT INTO SC VALUES ('2005030106','030202',98)
INSERT INTO T VALUES ('T00001','赵亮','男','教授','是')
INSERT INTO T VALUES ('T00002','李丽','女','讲师','否')
INSERT INTO T VALUES ('T00003','王海','男','教授','是')
INSERT INTO TC VALUES ('T00001','020101','优秀')
INSERT INTO TC VALUES ('T00002','010101','良好')
INSERT INTO TC VALUES ('T00002','030101','一般')
INSERT INTO TC VALUES ('T00003','030102','一般')
INSERT INTO TC VALUES ('T00003','030201','较差')
INSERT INTO TC VALUES ('T00003','030202','优秀')
```

```
GO
SELECT * FROM S WHERE SSex='女'
SELECT SName,Birth,Coll FROM S,M
    WHERE Birth NOT BETWEEN '1997-1-1' AND '1998-1-1' AND
    S.MNo=M.MNo
SELECT SName,Birth,Coll FROM S,M
    WHERE (Birth <'1997-1-1' OR Birth >'1998-1-1') AND
    S.MNo=M.MNo
SELECT SName FROM S,M
    WHERE S.MNo=M.MNo AND Coll IN ('信息学院','外语学院')
SELECT SName FROM S,M
    WHERE S.MNo=M.MNo AND (Coll='信息学院' OR Coll='外语学院')
SELECT * FROM C WHERE CNo='020101'
SELECT * FROM C WHERE CNo LIKE '020101'
SELECT SName,SNo,SSex FROM S WHERE SName LIKE '马%'
SELECT SName FROM S WHERE SName LIKE '刘_鑫%'
SELECT * FROM C WHERE CName LIKE 'Java\_%y_' ESCAPE '\'
SELECT * FROM SC ORDER BY CNo,SNo DESC
SELECT CNo,MAX(Grade),MIN(Grade),AVG(Grade)
    FROM SC WHERE CNo='020101' GROUP BY CNo
SELECT CNo,COUNT(SNo) FROM SC GROUP BY CNo
SELECT SNo FROM SC GROUP BY SNo HAVING COUNT(*)>6
SELECT SNo FROM SC WHERE Grade>=90 GROUP BY SNo HAVING COUNT(*)>=6
SELECT T1.CNo,T2.CPNo
    FROM C T1,C T2
    WHERE T1.CPNo=T2.CNo
SELECT S.SNo,SName,CName,Grade
    FROM S,C,SC
    WHERE S.SNo=SC.SNo AND C.CNo=SC.CNo
SELECT S1.SNo,S1.SName FROM S S1,S S2
    WHERE S1.MNo=S2.MNo AND S2.SName='李光'
SELECT SNo,SName FROM S WHERE MNo IN
    (SELECT MNo FROM S WHERE SName='李光')
SELECT SNo,SName FROM S S1 WHERE EXISTS
```

```
    (SELECT * FROM S S2 WHERE S2.MNo=S1.MNo AND S2.SName='李光')
SELECT SName FROM S WHERE SNo IN
    (SELECT SNo FROM SC WHERE CNo IN
        (SELECT CNo FROM C WHERE CName='图像分析'))
SELECT SName FROM S S,C C,SC SC
    WHERE SC.SNo=S.SNo AND
          SC.CNo=C.CNo AND C.CName='图像分析'
SELECT SName FROM S WHERE MNo<>'M0002' AND Birth<ANY
    (SELECT Birth FROM S WHERE MNo='M0002')
SELECT SName FROM S WHERE MNo<>'M0002' AND Birth<
    (SELECT MAX(Birth) FROM S WHERE MNo='M0002')
SELECT SName FROM S WHERE MNo<>'M0003' AND Birth<ALL
    (SELECT Birth FROM S WHERE MNo='M0003')
SELECT SName FROM S WHERE MNo<>'M0003' AND Birth<
  (SELECT MIN(Birth) FROM S WHERE MNo='M0003')
SELECT SName FROM S S WHERE NOT EXISTS
    (SELECT * FROM C C WHERE NOT EXISTS
    (SELECT * FROM SC SC
        WHERE SC.SNo=S.SNo AND SC.CNo=C.CNo))
SELECT DISTINCT SNo FROM SC X WHERE NOT EXISTS
    (SELECT * FROM SC Y WHERE Y.SNo='2005010101' AND NOT EXISTS
    (SELECT * FROM SC Z WHERE Z.SNo=X.SNo AND Z.CNo=Y.CNo))
```

实验 7　用户管理

针对“实验 1”创建的 CInfo 和 SInfo，在 SSMS 中，分别使用对象资源管理器和查询分析器，实现如下任务：

（1）利用 Windows 7/8/10，创建三个 Windows 用户，用户名和密码如下：

WinTim、WinJim、WinMim；wtim123、wjim123、wmim123。

（2）创建和删除三个 Windows 身份的 SQL Server 登陆。即：

WinTim、WinJim、WinMim。

（3）创建和删除三个 SQL Server 身份的 SQL Server 登陆。用户名和密码如下：

SqlTim、SqlJim、SqlMim；stim123、sjim123、smim123。

（4）创建和删除访问 CInfo 的数据库用户 UWinTim、UWinJim、UWinMim，身份验证分别来自登陆 WinTim、WinJim、WinMim。

（5）创建和删除访问 SInfo 的数据库用户 USqlTim、USqlJim、USqlMim，身份验证分别来自登陆 SqlTim、SqlJim、SqlMim。

（6）创建和删除访问 CInfo 的数据库角色 WinRole，角色的成员分别为数据库用户 UWinTim、UWinJim、UWinMim。

（7）创建和删除访问 SInfo 的数据库角色 SqlRole，角色的成员分别为数据库用户 USqlTom、USqlJim、USqlMim。

登录用户

Windows 登录用户

开始→控制面板→用户帐户和家庭安全→用户帐户。

在 SSMS 的对象资源管理器中，展开要在其中创建登录的服务器实例的文件夹；右击“安全性”文件夹，指向“新建”，然后单击“登录名”；在“常规”页上的“登录名”框中直接输入一个 Windows 用户名或者通过“搜索”选择一个 Windows 用户名；选择“Windows 身份验证”；单击“确定”。

```
CREATE LOGIN [HAPPYU-ZJSU\WinTim] FROM WINDOWS
CREATE LOGIN [HAPPYU-ZJSU\WinJim] FROM WINDOWS
CREATE LOGIN [HAPPYU-ZJSU\WinMim] FROM WINDOWS
```

SQL Server 登录用户

在 SSMS 的对象资源管理器中，展开要在其中创建登录的服务器实例的文件夹；右击“安全性”文件夹，指向“新建”，然后单击“登录名”；在“常规”页上的“登录名”框中输入一个新登录名的名称；选择“SQL Server 身份验证”（Windows 身份验证是更安全的选择）；输入登录名的密码（选择应当应用于新登录名的密码策略选项，通常强制密码策略是更安全的选择）；单击“确定”。

```
CREATE LOGIN [SqlTim] WITH PASSWORD='stim123'
CREATE LOGIN [SqlJim] WITH PASSWORD='sjim123'
CREATE LOGIN [SqlMim] WITH PASSWORD='smim123'
```

数据库用户

在 SSMS 的对象资源管理器中，展开要在其中创建登录的服务器实例的文件夹；展开 CInfo 数据库，展开“安全性”，右击“用户”，指向“新建用户”，然后在“数据库用户-新建”界面上的“用户名”框中输入一个新用户名的名称；在“登录名”右侧单击“...”，单击“浏览”，勾选“WinTim”，单击“确定”，单击“确定”；单击“确定”。

同理：建立“WinJim”、“WinMim”。

同理：建立 SInfo 的相关数据库用户。

```
USE CInfo
CREATE USER UWinTim FOR LOGIN [HAPPYU-ZJSU\WinTim]
CREATE USER UWinJim FOR LOGIN [HAPPYU-ZJSU\WinJim]
CREATE USER UWinMim FOR LOGIN [HAPPYU-ZJSU\WinMim]
USE SInfo
CREATE USER USqlTim FOR LOGIN [SqlTim]
CREATE USER USqlJim FOR LOGIN [SqlJim]
CREATE USER USqlMim FOR LOGIN [SqlMim]
```

数据库角色

在 SSMS 的对象资源管理器中，展开要在其中创建登录的服务器实例的文件夹；展开 CInfo 数据库，展开“安全性”，右击“角色”，指向“新建”，指向“新建数据库角色”，然后在“数据库角色-新建”界面上的“角色名称”框中输入一个新角色的名称 WinRole；在“所有者”右侧单击“...”，单击“浏览”，勾选“dbo”，单击“确定”；

单击“添加”，在“选择数据库用户或角色”界面中，单击“浏览”，勾选“UWinTim”、“UWinJim”、“UWinMim”，单击“确定”，单击“确定”；单击“确定”。

同理：建立 SInfo 的相关数据库角色。

```
USE CInfo
CREATE ROLE WinRole AUTHORIZATION dbo
EXEC SP_ADDROLEMEMBER WinRole,'UWinTim'
EXEC SP_ADDROLEMEMBER WinRole,'UWinJim'
EXEC SP_ADDROLEMEMBER WinRole,'UWinMim'
USE SInfo
CREATE ROLE SqlRole AUTHORIZATION dbo
EXEC SP_ADDROLEMEMBER SqlRole,'USqlTim'
```

```
EXEC SP_ADDROLEMEMBER SqlRole,'USqlJim'
EXEC SP_ADDROLEMEMBER SqlRole,'USqlMim'
```

删除操作

删除数据库角色及其成员

```
USE CInfo
EXEC SP_DROPROLEMEMBER 'WinRole', 'UWinMim'
EXEC SP_DROPROLEMEMBER 'WinRole', 'UWinJim'
EXEC SP_DROPROLEMEMBER 'WinRole', 'UWinTim'
DROP ROLE WinRole
```

删除数据库用户

```
DROP USER UWinMim
```

删除登陆用户

```
DROP LOGIN [HAPPYU-ZJSU\WinMim]
```

实验 8　授权与收权

针对“实验 1”创建的 CInfo 和 SInfo，在 SSMS 中，分别使用对象资源管理器和查询分析器，实现如下任务：

（1）把查询商品的权限授给用户 UWinTim。

（2）把对商品,商店的全部权限授予用户 UWinJim 和 UWinMim。

（3）把对商品的查询权限授予所有用户。

（4）把查询商品和修改品号的权限授给用户 WinRole。

（5）把对商店的 INSERT 权限授予 UWinMim，并允许他再授予 UWinJim。

（6）DBA 把在数据库 CInfo 中建立表的权限授予用户 UWinJim。

（7）阻止 UWinTim 和 UWinMim 的 CREATE TABLE 和 CREATE PROCEDURE 权限。

（8）阻止用户 UWinTim 和 UWinMim 对商品的读取权限。

（9）收回 WinRole 修改品号的权限。

（10）针对 SInfo，设计一套数据库用户（USqlTim、USqlJim、USqlMim）和数据库角色 WinRole 的授权方案。

CInfo 权限管理：

```
USE CInfo
GRANT SELECT ON Ware TO UWinMim
GRANT ALL PRIVILEGES ON Ware TO UWinMim, UWinTim
GRANT ALL PRIVILEGES ON Shop TO UWinMim, UWinJim
GRANT SELECT ON Ware TO PUBLIC
GRANT UPDATE(WNo),SELECT ON Ware TO WinRole
GRANT INSERT ON Shop TO UWinMim WITH GRANT OPTION
GRANT INSERT ON Shop TO UWinJim
GRANT CREATE TABLE TO UWinJim
DENY CREATE TABLE,CREATE PROCEDURE TO UWinTim, UWinMim
DENY SELECT ON Ware TO UWinTim, UWinMim
REVOKE UPDATE(WNo) ON Ware FROM WinRole
```

SInfo 权限管理：请自行完成。

使用对象资源管理器的权限管理：请自行完成。

实验 9　视图

针对“实验 1”创建的 CInfo 和 SInfo，在 SSMS 中，分别使用对象资源管理器和查询分析器，实现如下任务：

（1）利用 SInfo，建立信息学院学生的详细信息的“信息学院”视图。

（2）利用 SInfo，建立包含姓名、性别、课程名和成绩等信息的“成绩信息”视图。

（3）利用 SInfo，建立选修“图像分析”的学生的学号、姓名、课程名和成绩的视图“图像分析成绩”。

（4）利用 SInfo，修改“信息学院”视图，把学生性别限制为男生。

（5）利用“信息学院”视图，查询信息学院的男生信息。

（6）利用“信息学院”视图，建立信息学院男生的学号、姓名和生日信息的“信息学院男生”视图。

（7）向“信息学院男生”视图中，添加如下记录：

('2005090109','海洋','1999-09-09')

（8）在“信息学院男生”视图中，把刘金改为刘鑫。

（9）在“信息学院男生”视图中，把海洋的记录删除。分析不能删除的原因！

（10）利用“成绩信息”视图，查询小于等于 95 的学生的姓名、课程名和成绩。

（11）利用“图像分析成绩”视图，查询图像分析课程的最高分、最低分和平均分。

（12）删除“图像分析成绩”视图。

（13）针对 CInfo，设计若干视图，并对视图进行相应的插入、修改、查询和删除等操作。

使用对象资源管理器：具体视图操作，请自行完成。

启动查询设计器（SSMS→对象资源管理器→展开学籍→右击视图→新建视图）。

或者从菜单中启动查询设计器（查询→在编辑器中设计查询）。

新建视图 / 修改视图 / 运行视图 / 删除视图。

使用查询分析器：

```
USE SInfo
GO
CREATE VIEW 信息学院 AS
    SELECT S.SNo,SName,SSex,Birth,Coll,C.CNo,CName,CPNo,Credit,Grade
    FROM S, C, SC, M
    WHERE S.SNo=SC.SNo AND S.MNo=M.MNo AND
        SC.CNo=C.CNo AND Coll='信息学院'
GO
CREATE VIEW 成绩信息 AS
```

```
        SELECT SName,SSex,CName,Grade
        FROM S, C, SC
        WHERE S.SNo=SC.SNo AND C.CNo=SC.CNo
    GO
    CREATE VIEW 图像分析成绩(SNo,SName,CName,Grade) AS
        SELECT S.SNo,SName,CName,Grade
        FROM S,C,SC
        WHERE S.SNo=SC.SNo AND C.CNo=SC.CNo AND CName='图像分析'
    GO
    ALTER VIEW 信息学院 AS
        SELECT S.SNo,SName,SSex,Birth,Coll,C.CNo,CName,CPNo,Credit,Grade
        FROM S, C, SC, M
        WHERE S.SNo=SC.SNo AND S.MNo=M.MNo AND
            SC.CNo=C.CNo AND Coll='信息学院' AND SSex='男'
    GO
    SELECT * FROM 信息学院
    GO
    CREATE VIEW 信息学院男生 AS
        SELECT SNo,SName,Birth
        FROM 信息学院
    GO
    INSERT INTO 信息学院男生 VALUES ('2005090109','海洋','1999-09-09')
    UPDATE 信息学院男生 SET SName='刘鑫'
        WHERE SName='刘金'
    DELETE FROM 信息学院男生 WHERE SName='海洋'
    SELECT SName,CName,Grade
        FROM 成绩信息
        WHERE Grade<=95
    SELECT CName, MAX(Grade), MIN(Grade), AVG(Grade)
        FROM 图像分析成绩
        GROUP BY CName
    DROP VIEW 图像分析成绩
```

CInfo 视图操作：请自行完成。

实验 10　并发控制

在“实验 1”创建的 CInfo 和 SInfo 中，事务回滚和提交的实现；并发事务的丢失修改、读脏数据和不可重读问题的测试和解决方法。

（1）分析并写出如下语句的含义和执行结果。

```
USE Master
CREATE TABLE TestTable (Attr CHAR(6))
GO
DECLARE @TranName1 VARCHAR(10) = 'TranRoback'
DECLARE @TranName2 VARCHAR(10) = 'TranCommit'
BEGIN TRAN @TranName1
    INSERT INTO TestTable VALUES('Hello')
    INSERT INTO TestTable VALUES('World!')
ROLLBACK TRAN @TranName1
BEGIN TRAN @TranName2
    INSERT INTO TestTable VALUES('Happy')
    INSERT INTO TestTable VALUES('You!')
COMMIT TRAN @TranName2
SELECT * FROM TestTable
DROP TABLE TestTable
```

（2）SQL Server 2016 的自动封锁测试。

在 CInfo 中，使用 SQL 语句实现如下功能，然后写出执行结果。同时分析对修改操作（UPDATE），DBMS 会自动在何种粒度的数据对象上添加何种类型的封锁？

```
-- 打开数据库 CInfo
-- 定义一个事务 EditName
-- 把厂商“Sony 公司”的名称改为“Sony 有限公司”
-- 查看数据库的 CInfo 封锁信息（提示：SP_LOCK）
-- 回滚事务 EditName
-- 查看数据库的 CInfo 封锁信息
```

（3）丢失修改的用户封锁测试和解决方法。

在 CInfo 中，使用 SQL 语句实现如表 10.1 所示的并发事务 T1 和 T2 的功能，然后写出执行结果。同时分析丢失修改的原因，最后给出解决问题的方法。

表 10.1　丢失修改的实现

T1	T2
-- 打开数据库 CInfo	-- 打开数据库 CInfo
-- 定义一个事务 T1	-- 定义一个事务 T2
-- 声明整型变量@Var	-- 声明整型变量@Var
-- 读取张三的工资到@Var	-- 读取张三的工资到@Var
-- 等待 10 秒	-- 等待 10 秒
-- 用@Var+600 更新张三的工资	-- 用@Var-500 更新张三的工资
提交事务 T1	提交事务 T2

提示 1：利用两个查询分析窗口，模拟两个连接到 SQL Server 服务器的客户端，首先在窗口 1 中运行事务 T1，然后立即在窗口 2 中运行事务 T2。

提示 2：修改 T1 如下：

```
SELECT @Var=Salary
FROM Emp WITH (TABLOCKX)
WHERE EName= '张三'
```

（4）读脏数据的用户封锁测试和解决方法。

在 CInfo 中，使用 SQL 语句实现如表 10.2 所示的并发事务 T1 和 T2 的功能，然后写出执行结果。同时分析读脏数据的原因，最后给出解决问题的方法。

表 10.2　读脏数据的实现

T1	T2
-- 打开数据库 CInfo	-- 打开数据库 CInfo
-- 定义一个事务 T1	-- 定义一个事务 T2
-- 声明整型变量@Var	-- 声明整型变量@Var
-- 读取张三的工资到@Var	-- 读取张三的工资到@Var
-- 用@Var+600 更新张三的工资	-- 如果@Var 不等于 5000，提示“读脏数据”
-- 等待 10 秒	提交事务 T2
回滚事务 T1	

提示 1 :如果事务当前的隔离级别是默认的 READ COMMITTED ,则不会出现“读脏数据”的问题。

提示 2：修改 T2 如下：

```
SELECT @Var=Salary
FROM Emp WITH (HOLDLOCK)
WHERE EName= '张三'
```

提示 3：如果采用事务隔离级别控制，需在 T1 开始之前设置：

SET TRANSACTION ISOLATION LEVEL READ COMMITTED

（5）不可重读的用户封锁测试和解决方法。

在 CInfo 中，使用 SQL 语句实现如表 10.3 所示的并发事务 T1 和 T2 的功能，然后写出执行结果。同时分析读脏数据的原因，最后给出解决问题的方法。

表 10.3　不可重读的实现

T1	T2
-- 打开数据库 CInfo	-- 打开数据库 CInfo
-- 定义一个事务 T1	-- 定义一个事务 T2
-- 声明整型变量@Var1，@Var2	-- 声明整型变量@Var
-- 读取张三的工资到@Var1	-- 读取张三的工资到@Var
-- 等待 10 秒	
-- 读取张三的工资到@Var2	-- 用@Var+600 更新张三的工资
-- 如果@Var1 不等于@Var2，提示“不可重读!”	提交事务 T2
提交事务 T1	

提示 1：修改 T1 如下：

SELECT @Var=Salary

FROM Emp WITH (HOLDLOCK)

WHERE EName= '张三'

提示 2：如果采用事务隔离级别控制，需在 T1 开始之前设置：

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ

（6）在 SInfo 中，自行设计并实现事务的回滚和提交，并发事务的丢失修改、读脏数据和不可重读问题的验证和解决方案。

提示：HOLDLOCK（共享锁），NOLOCK（不加锁），ROWLOCK（行级锁），TABLOCK（表级共享锁锁），TABLOCKX（表级排它锁）， READ [UN]COMMITTED（隔离级别），REPEATABLE READ（隔离级别），SERIALIZABLE（隔离级别），@@LOCK_TIMEOUT （锁超时设置），WAITFOR DELAY（时间延迟），SP_LOCK（看锁信息），UPDLOCK（更新锁）等。

参考实验方案：

（1）请自行分析和验证。

（2）

-- 打开数据库 CInfo

USE CInfo

-- 定义一个事务 EditName

BEGIN TRANSACTION EditName

-- 把厂商“Sony 公司”的名称改为“Sony 有限公司”

UPDATE Ware SET Maker='Sony 有限公司' WHERE Maker='Sony 公司'

-- 查看数据库的 CInfo 封锁信息（提示：SP_LOCK）

EXECUTE SP_LOCK

-- 回滚事务 EditName

ROLLBACK TRANSACTION EditName

-- 查看数据库的 CInfo 封锁信息

EXECUTE SP_LOCK

执行结果与分析：请自行完成。

（3）实现方案如表 10.4 所示。

表 10.4　丢失修改的实现

T1	T2
USE CInfo	USE CInfo
BEGIN TRAN T1	BEGIN TRAN T2
DECLARE @Var INT	DECLARE @Var INT
SELECT @Var=工资	SELECT @Var=工资
FROM 职工	FROM 职工
WHERE 姓名='张三'	WHERE 姓名='张三'
WAITFOR DELAY '00:00:10'	WAITFOR DELAY '00:00:10'
UPDATE 职工	UPDATE 职工
SET 工资=@Var+600	SET 工资=@Var-500
WHERE 姓名='张三'	WHERE 姓名='张三'
COMMIT TRAN T1	COMMIT TRAN T2

（4）实现方案如表 10.5 所示。

表 10.5　读脏数据的实现

T1	T2
USE CInfo	USE CInfo
BEGIN TRAN T1	BEGIN TRAN T2
DECLARE @Var INT	DECLARE @Var INT
SELECT @Var=工资	SELECT @Var=工资
FROM 职工	FROM 职工
WHERE 姓名='张三'	WHERE 姓名='张三'
UPDATE 职工	IF (@Var<>5000) PRINT '读脏数据!'
SET 工资=@Var+600	COMMIT TRAN T2
WHERE 姓名='张三'	
WAITFOR DELAY '00:00:10'	
ROLLBACK TRAN T1	

（5）实现方案如表 10.6 所示。

表 10.6　不可重读的实现

T1	T2
USE CInfo	USE CInfo
BEGIN TRAN T1	BEGIN TRAN T2
DECLARE @Var1 INT,@Var2 INT	DECLARE @Var INT
SELECT @Var1=工资	SELECT @Var=工资
FROM 职工 WITH	FROM 职工
WHERE 姓名='张三'	WHERE 姓名='张三'
WAITFOR DELAY '00:00:10'	
SELECT @Var2=工资	UPDATE 职工
FROM 职工 WITH	SET 工资=@Var+600
WHERE 姓名='张三'	WHERE 姓名='张三'
IF (@Var1<>@Var2) RAISERROR('不可重读!',16,1)	COMMIT TRAN T2
ROLLBACK TRAN T1	

（6）在 SInfo 中，并发事务的控制设计与实现：请自行完成。

实验 11　数据恢复

在“实验 1”创建的 CInfo 和 SInfo 中，分别使用对象资源管理器和查询编辑器，实现数据库的备份与还原、分离和附加、导入与导出等。

（1）备份与还原。把 CInfo 按照完整、差异、事务日志和镜像等备份到指定目录；然后，再把备份文件中的数据，还原到当前数据库中。

提示 1：BACKUP DATABASE CInfo TO DISK='D:\Data\CInfo.bak'

提示 2：RESTORE DATABASE CInfo FROM DISK='D:\Data\CInfo.bak' WITH REPLACE

（2）SQL Server 与 Access 的导入与导出。把 CInfo 的职工表，导出到指定的 Access 数据库中，表名不变。同时，把 Access 数据库中职工表导入到 SInfo 中，表名不变。

（3）SQL Server 与 Excel 的导入与导出。把 CInfo 的商品表，导出到指定的 Excel 文件中，表名不变。同时，把 Excel 文件中的商品表导入到 SInfo 中，表名不变。

（4）SQL Server 与文本文件的导入与导出。把 CInfo 的商店表，导出到商店.txt 中。同时，把商店.txt 导入到 SInfo 中，表名不变。

（5）分离与附加。把 CInfo 从当前服务器中分离出来，然后对分离后的数据库及其日志文件进行备份，最后再把分离的数据库文件，附加到当前服务器中。

提示 1：SP_DETACH_DB CInfo

提示 2：
```
EXEC SP_ATTACH_DB @DBNAME='CInfo',
                  @FILENAME1='CInfo.mdf',
                  @FILENAME2='CInfo.ldf'
```

参考实验方案：

（1）

```
--将测试数据库的恢复模式设置为 FULL
ALTER DATABASE CInfo SET RECOVERY FULL
BACKUP DATABASE CInfo TO DISK='D:\Data\CInfoFull.bak'
    WITH FORMAT
BACKUP DATABASE CInfo TO DISK='D:\Data\CInfoDiff.bak'
    WITH FORMAT, DIFFERENTIAL
--日志备份，不截断日志
BACKUP LOG CInfo TO DISK='D:\Data\CInfoLog.bak'
    With FORMAT, No_Truncate
BACKUP DATABASE CInfo TO
    DISK='D:\Data\CInfoMirr1.bak'
    MIRROR
```

```
    TO DISK='D:\Data\CInfoMirr2.bak'
    WITH FORMAT
RESTORE DATABASE CInfo FROM DISK='D:\Data\CInfoFull.bak'
    WITH REPLACE, NORECOVERY
RESTORE DATABASE CInfo FROM DISK='D:\Data\CInfoDiff.bak'
    WITH NORECOVERY
RESTORE DATABASE CInfo FROM DISK='D:\Data\CInfoLog.bak'
    WITH NORECOVERY
RESTORE DATABASE CInfo FROM DISK='D:\Data\CInfoMirr1.bak'
    WITH NORECOVERY
RESTORE DATABASE CInfo FROM DISK='D:\Data\CInfoMirr2.bak'
    WITH RECOVERY
```

（2）请自行完成。

（3）请自行完成。

（4）请自行完成。

（5）

```
SP_DETACH_DB CInfo
-- 使用 Windows 中的复制和粘贴，对分离后的数据库进行备份。
EXEC SP_ATTACH_DB @DBNAME='CInfo',
    @FILENAME1='D:\MyData\ComInfo.mdf',
    @FILENAME2='D:\MyData\ComInfo.ldf'
```

使用对象资源管理器的数据备份与恢复操作：请自行完成。

实验 12　SQL Server 程序设计

（1）设计程序输出三位数中的素数。

```
DECLARE @i int,@j int
SET @i=100
WHILE (@i<=999)
BEGIN
    SET @j=2
    WHILE (@j<@i)
    BEGIN
        IF (@i%@j=0)
            BREAK
        SET @j=@j+1
    END
    IF (@j=@i)
        PRINT @i
    SET @i=@i+1
END
```

（2）设计程序输出百钱买百鸡的购买方案。

小鸡 1 元 3 只，公鸡 3 元 1 只，母鸡 5 元 1 只，输出 100 元买 100 只鸡的购买方案。

方法 1：

```
DECLARE @i int,@j int,@k int
SELECT @i=0,@j=0,@k=0
WHILE (@i<=100/5)
BEGIN
    WHILE (@j<=100/3)
    BEGIN
        WHILE (@k<=100/1)
        BEGIN
            IF ((@i*5+@j*3+@k/3*1=100) AND (@i+@j+@k=100))
                SELECT @i as Male,@j as Female ,@k as Child
            SET @k=@k+1
        END
        SELECT @j=@j+1,@k=0
```

```
        END
        SELECT @i=@i+1,@j=0,@k=0
END
```

方法 2：

```
DECLARE @i int,@j int
SET @i=0
WHILE (@i<=20)
BEGIN
      WHILE (@j<=33)
      BEGIN
            IF (@i*5+@j*3+(100-@i-@j)/3*1=100)
                  SELECT @i as Male,@j as Female,(100-@i-@j) as Child
            SELECT @j=@j+1
        END
        SELECT @i=@i+1,@j=0
END
```

（3）设计程序输出水仙花数。水仙花数是满足如下公式的三位数（例如：153）。

$$xyz = x^3 + y^3 + z^3$$

```
DECLARE @i int,@j int,@k int
SET @i=1
WHILE (@i<=9)
BEGIN
      SET @j=0
      WHILE (@j<=9)
      BEGIN
            SET @k=0
            WHILE (@k<=9)
            BEGIN
                  IF (POWER(@i,3)+POWER(@j,3)+POWER(@k,3) =
                        @i*100+@j*10+@k)
                         SELECT @i*100+@j*10+@k
                  SET @k=@k+1
            END
            SET @j=@j+1
```

```
    END
    SET @i=@i+1
END
```

实验 13　游标

（1）利用游标输出每一本图书的书号、书名、作者、销量和利润。格式如下：

书号：ISBN978-7-04-040664-1；书名：数据库系统概论；作者：王珊；销量：2；利润：10

……

参考代码：

```
DECLARE @BNo CHAR(22),@BName CHAR(22),@Author CHAR(8),@SVol INT,@Profit FLOAT
DECLARE BProCur CURSOR FOR
    SELECT Book.BNo  书号,BName  书名,Author  作者,
        COUNT(CNo)  销量, SUM(Sprice-Pprice)  利润
    FROM Buy,Book
    WHERE Book.BNo=Buy.BNo
    GROUP BY Book.BNo,BName,Author
OPEN BProCur
FETCH NEXT FROM BProCur INTO @BNo,@BName,@Author,@SVol,@Profit
WHILE @@FETCH_STATUS = 0
    BEGIN
        PRINT '书号：'+@BNo+'书名：'+@BName+'作者：'+@Author+
              '销量：'+CAST( @SVol AS VARCHAR)+
              '   利润：'+CONVERT(VARCHAR(6), @Profit)
        FETCH NEXT FROM BProCur INTO @BNo,@BName,@Author,@SVol,@Profit
    END
CLOSE BProCur
DEALLOCATE BProCur
```

（2）利用游标输出每个客户购书清单。格式如下：

户号：C001；户名：李明；性别：男；电话：13611122233

书号	书名	作者	社名
ISBN978-7-04-040664-1	数据库系统概论	王珊	高等教育出版社
ISBN978-7-5612-2591-2	图像技术	韩培友	西北工业大学出版社

……

户号：C002；户名：吴光；性别：男；电话：13911122233

书号	书名	作者	社名
ISBN978-7-04-040664-1	数据库系统概论	王珊	高等教育出版社
ISBN978-7-302-33894-9	Java 程序设计教程	雍俊海	清华大学出版社

……

嵌套游标格式：

```
--声明外层游标提取的变量
--声明内层游标提取的变量
--声明外层游标
--打开外层游标
--提取外层游标第一行数据
--用 while 循环遍历外层游标
--BEGIN
    --对外层游标提取的数据进行处理
    /*
    --声明内层游标
    --打开内层游标
    --提取内层游标第一行数据
    --用 while 循环遍历内层游标
    --BGEIN
      --对内层游标提取的数据进行处理
      --提取内层游标下一行数据
    --END
    --关闭内层游标
    --释放内存游标
    */
    --提取外层游标下一行数据
    --对外层游标提取的数据进行后处理
--END
--关闭外层游标
--释放外层游标
```

参考代码：

```
DECLARE @CNo CHAR(4),@CName NVARCHAR(8),@CSex CHAR(2),@Phone CHAR(11)
DECLARE @BNo CHAR(22),@BName CHAR(22),@Author CHAR(8),@PName CHAR(16)
DECLARE CusInfoCur CURSOR FOR
  SELECT CNo,CName,CSex,Phone FROM Cust
OPEN CusInfoCur
FETCH NEXT FROM CusInfoCur INTO @CNo,@CName,@CSex,@Phone
```

```
WHILE @@FETCH_STATUS=0
  BEGIN
    PRINT '户号：'+@CNo+'  户名：'+@CName+'  性别：'+@CSex+'  电话：'+@Phone
    PRINT '书号'+SPACE(19)+'书名'+SPACE(19)+'作者'+SPACE(5)+'社名'
    DECLARE BooInfoCur CURSOR FOR
      SELECT Buy.BNo,BName,Author,PName
      FROM Buy,Book,Press
      WHERE Buy.CNo=@CNo AND Buy.BNo=Book.BNo AND Book.PNo=Press.PNo
    OPEN BooInfoCur
    FETCH NEXT FROM BooInfoCur INTO @BNo,@BName,@Author,@PName
    WHILE @@FETCH_STATUS=0
      BEGIN
        PRINT @BNo+' '+@BName+' '+@Author+' '+@PName
        FETCH NEXT FROM BooInfoCur INTO @BNo,@BName,@Author,@PName
      END
    CLOSE BooInfoCur
    DEALLOCATE BooInfoCur
    PRINT ''
    FETCH NEXT FROM CusInfoCur INTO @CNo,@CName,@CSex,@Phone
  END
CLOSE   CusInfoCur
DEALLOCATE CusInfoCur
```

实验 14　存储过程和函数

（1）已知三个数 a、b 和 c，利用存储过程返回三个数的升序排序结果 x、y 和 z。

参考代码：

```
CREATE PROC Pro1
    @a INT,
    @b INT,
    @c INT,
    @x INT OUT,
    @y INT OUT,
    @z INT OUTPUT
AS
    BEGIN
        SET @x=@a
        SET @y=@b
        SET @z=@c
        IF @x>@y
            BEGIN
                SET @a=@x
                SET @x=@y
                SET @y=@a
            END
        IF @x>@z
            BEGIN
                SET @a=@x
                SET @x=@z
                SET @z=@a
            END
        IF @y>@z
            BEGIN
                SET @a=@y
                SET @y=@z
                SET @z=@a
            END
```

```
    END
GO
DECLARE @a INT,@b INT,@c INT,@x INT,@y INT,@z INT
SELECT @a=6,@b=3,@c=9
EXEC Pro1
    @a, @b, @c,
    @x OUT,@y OUT,@z OUT
SELECT @x,@y,@z
```

（2）已知三个数 a、b 和 c，利用存储过程返回三个数的最小、最大、和值和均值。

参考代码：

```
CREATE PROC Pro2
    @a INT,
    @b INT,
    @c INT,
    @Mi INT OUT,
    @Ma INT OUT,
    @Su INT OUT,
    @Av FLOAT OUTPUT
AS
    BEGIN
        SET @Mi=@a
        SET @Ma=@a
        IF @b>@Ma SET @Ma=@b
        IF @c>@Ma SET @Ma=@c
        IF @b<@Mi SET @Mi=@b
        IF @c<@Mi SET @Mi=@c
        SET @Su=@a+@b+@c
        SET @Av=(@a+@b+@c)/3.0
    END
GO
DECLARE @a INT,@b INT,@c INT,@Mi INT,@Ma INT,@Su INT,@Av FLOAT
SELECT @a=16,@b=6,@c=19
EXEC Pro2
    @a, @b, @c,
```

```
    @Mi OUT,@Ma OUT,@Su OUT,@Av OUT
SELECT @Mi,@Ma,@Su,CAST(@Av AS DECIMAL(6,2))
```

（3）已知三角形的三边分别为 a、b 和 c，利用函数计算三角形的面积 Area。

$$Area = \sqrt{s(s-a)(s-b)(s-c)}$$

$$s = (a+b+c)/2$$

参考代码：

```
CREATE FUNCTION TriAre(@a REAL,@b REAL,@c REAL)
    RETURNS REAL
AS
    BEGIN
        DECLARE @s NUMERIC(9,2)
        SET @s=(@a+@b+@c)/2.0
        SET @s=SQRT(@s*(@s-@a)*(@s-@b)*(@s-@c))
        RETURN @s
    END
GO
DECLARE @a REAL,@b REAL,@c REAL,@Sum NUMERIC(9,2)
SELECT @a=3,@b=6,@c=5
SET @Sum=DBO.TriAre(@a,@b,@c)
SELECT @Sum
```

（4）设计并实现如下函数。

$$f(x) = \begin{cases} x^2 + xCosx + 9 & x < -6 \\ 0 & |x| \le 6 \\ x^2 + xSinx - 9 & x > 6 \end{cases}$$

参考代码：

```
CREATE FUNCTION Fx(@x REAL)
    RETURNS REAL
AS
    BEGIN
        DECLARE @y NUMERIC(9,2)
        IF @x<-6
            BEGIN
                SET @y=POWER(@x,2)+@x*COS(@x)+9.0
            END
        ELSE IF @x<=6
```

```
            BEGIN
                SET @y=0.0
            END
        ELSE
            BEGIN
                SET @y=POWER(@x,2)+@x*SIN(@x)-9.0
            END
        RETURN @y
    END
GO
DECLARE @x REAL,@y NUMERIC(9,2)
SELECT @x=9
SET @y=DBO.Fx(@x)
SELECT @y
```

实验 15　数据库设计与实现

1．利用 VC++和 SQL Server，实现例 10.17 中的功能，并进行完善。

2．使用选定的主语言和 DBMS，设计并实现具有如下功能的电子书店 EBookInfo：

（1）四张表 Book、Cust、Buy 和 Press 中数据的添加、修改和删除等功能。

（2）四张表中基本数据的查询功能。

（3）EBook 中进货和销售等信息的基本统计功能。

（4）其它信息管理功能自定。

参考实验素材：

1．

根据如下参考代码，请自行完成。

```
/*********************************************************************/
#import "c:\Program Files\Common Files\System\ADO\msado15.dll"\
    no_namespace rename("EOF","EndOfFile")
#include <iostream>
#include <iomanip>          // 设置 setw()
using namespace std;
class STU
{   public:
        char sno[10];      // 学号
        char sname[20];   // 姓名
        char ssex[2];      // 姓别
        long sage;          // 年龄
        char coll[40];     // 学院
    public:
        STU(){}
        ~STU(){}
};
int main()
{   STU studt;
    // 初始化 OLE/COM 库环境，准备访问 ADO 接口
    ::CoInitialize(NULL);
    _ConnectionPtr m_pConnection("ADODB.Connection");
    _RecordsetPtr m_pRecordset("ADODB.Recordset");
```

```
_bstr_t bstrSQL("select * from StudentInfo");     // 查询语句
char * UseDB = "USE VcSqlAdoDB";   //打开数据库语句
char * query_cmd = "DELETE FROM StudentInfo WHERE sname='HappyTuple'";
char Yn;
try
{    // 创建 Connection 对象
     m_pConnection.CreateInstance("ADODB.Connection");
     // 设置连接字符串，必须是 BSTR 类型或者 _bstr_t 类型
      _bstr_t strConnect="DSN=VCSQLServer2005;Provider=MSDASQL;\
     Server=(local); uid=sa; pwd=123; ";
     // 如果是网络数据库，则 Server 形如 (192.168.1.5,3340)
     // 用户 sa 和密码 123 是数据库的授权访问密码
     m_pConnection->Open(strConnect,"","",adModeUnknown);
     if(m_pConnection==NULL)
          cerr << "Link data ERROR!\n";
     m_pConnection->Execute(UseDB,NULL,1); //打开数据库
     // 创建记录集对象
     m_pRecordset.CreateInstance(__uuidof(Recordset));
     // 读取表中记录，放入内存缓冲区(即：SQL 通讯区，SQLCA)
     // 同时把记录集对象指向 SQLCA(即：SQL Communication Area)
     m_pRecordset->Open(bstrSQL,m_pConnection.GetInterfacePtr(),
          adOpenDynamic,adLockOptimistic,adCmdText);
     _variant_t vsno,vsname,vssex,vsage,vcoll;
     // 对应数据库中表的 sno,sname,ssex,sage,coll
     cout << "学号        姓名        性别        年龄      学院";
     cout<< "\n----------------------------------------------\n";
     while (!m_pRecordset->EndOfFile)
     {   // 可以使用字段编号或者字段名
        vsno    =m_pRecordset->GetCollect(_variant_t((long)0));
        vsname=m_pRecordset->GetCollect("sname");
        vssex =m_pRecordset->GetCollect("ssex");
        vsage =m_pRecordset->GetCollect("sage");
        vcoll =m_pRecordset->GetCollect("coll");
        if (vsno.vt   != VT_NULL && vsname.vt != VT_NULL &&
```

```
    vsage.vt != VT_NULL && vssex.vt   != VT_NULL &&
    vcoll.vt != VT_NULL)
    {   cout.setf(ios::left);
        cout << setw(12) << (char*)(_bstr_t)vsno;
        cout << setw(8) << (char*)(_bstr_t)vsname;
        cout << setw(4) << (char*)(_bstr_t)vssex;
        cout << setw(4) << vsage.lVal;
        cout << setw(8) << (char*)(_bstr_t)vcoll;
        cout.unsetf(ios::left);
        cout << endl;
    }
    m_pRecordset->MoveNext(); // 记录指针(游标)移到下一条记录
}
cout << "\n----------------------------------------------\n";
cout << "\n 请输入姓名为: 'HappyTuple' 的学生信息\n";
cout << "\n 学号:"; cin   >> studt.sno;
cout << "\n 姓名:"; cin   >> studt.sname;
cout << "\n 年龄:"; cin   >> studt.sage;
cout << "\n 性别:"; cin   >> studt.ssex;
cout << "\n 学院:"; cin   >> studt.coll;
m_pRecordset->MoveFirst(); // 移动记录指针(游标)到第一条记录
m_pRecordset->AddNew();      // 添加新记录
m_pRecordset->PutCollect("sno"   , _variant_t(studt.sno   ));
m_pRecordset->PutCollect("sname", _variant_t(studt.sname));
m_pRecordset->PutCollect("ssex" , _variant_t(studt.ssex ));
m_pRecordset->PutCollect("sage" , _variant_t(studt.sage ));
m_pRecordset->PutCollect("coll" , _variant_t(studt.coll ));
m_pRecordset->Update();
cout << "\n 确定删除 'HappyTuple(快乐一族)(Y/N)? '"; cin   >> Yn;
if (Yn=='Y' || Yn=='y')
{    // 用 Execute 执行 SQL 语句删除满足条件的记录
     m_pConnection->Execute(query_cmd,NULL,1);
}
m_pRecordset->Close();   // 关闭记录集对象
```

```
    }
    // 捕捉异常
    catch(_com_error e)
    {    // 显示错误信息
        cerr << "\n ERROR: " << (char*)e.Description(); // 输出异常
    }
    if(m_pConnection->State)
        m_pConnection->Close();
    ::CoUninitialize();
    return 0;
}
/*****************************************************************/
```

2．根据例 10.18，请自行完成。

```
# UserInfoDBMS
# 代码: GB2312
'''
用户信息管理系统
    1.显示用户
    2.查找用户
    3.修改用户
    4.删除用户
    5.添加用户
    6.重置用户
用户信息:户号(uid)+密码(pwd)
用户数据:SQLite 数据库 userinfo.db+表 user
'''
try:#添加异常处理
    #显示用户信息
    def dispall():
        try:
            import sqlite3
            cn=sqlite3.connect('userinfo.db')
            cur=cn.execute('select * from user')
            user=cur.fetchall()
```

```
        if len(user)==0:
            print('\t 没有用户!')
        else:
            print('\t 用户信息: ')
            n=0
            for x in user:
                for a in x:
                    print('\t ',a,end='\t')
                print()
        cn.close()
        input('\n 回车继续...\n')
    except Exception as ex:
        print('\t 访问数据错误：',ex)
        raise ex
#查找/修改/删除
def seek_update():
    try:
        import sqlite3
        cn=sqlite3.connect('userinfo.db')
        uid=input('\t 输入户号:')
        if find(uid)==-1:
            print('\t%s  查无此人!' %uid)
        else:
            print('\t1.查询用户')
            print('\t2.修改用户')
            print('\t3.删除用户')
            op=input('\t 请选择：')
            if op=='1':
                #查询用户
                cur=cn.execute('select * from user where uid=?',(uid,))
                tm=cur.fetchone()
                print('\n\t 查询结果:\n')
                print('\t 户号: %s\n' %tm[0])
                print('\t 密码: %s\n' %tm[1])
```

```
                print('\n\t 查询结束!')
            elif op=='2':
                #修改用户
                nid=input('\t 输入新户名:')
                if nid=='':
                    print('\t 户名无效!')
                else:
                    #检查户号是否存在
                    if find(nid)==1:
                        print('\t 户号已经使用!')
                    else:
                        pwd=input('\t 输入新密码:')
                        if pwd=='':
                            print('\t 密码无效!')
                        else:
                            cn.execute('update user set uid=?,pwd=? where
uid=?',(nid,pwd,uid))
                            cn.commit()
                            print('\n\t 修改成功!')
            else:
                #删除用户
                cn.execute('delete from user where uid=?',(uid,))
                cn.commit()
                print('\n\t 删除成功!')

        cn.close()
        input('\n 回车继续...\n')
    except Exception as ex:
        print('\t 访问数据错误：',ex)
        raise ex
#添加用户
def adduser():
    try:
        import sqlite3
```

```
        cn=sqlite3.connect('userinfo.db')
        uid=input('\t 输入户号:')
        if uid=='':
            print('\t 户号无效!')
        else:
            #检查户号是否存在
            if find(uid)==1:
                print('\t 户号存在，重新添加!')
            else:
                pwd=input('\t 输入密码:')
                if pwd=='':
                    print('\t 密码无效!')
                else:
                    cn.execute('insert into user values(?,?)',(uid,pwd))
                    cn.commit()
                    print('\t 添加成功!')
        cn.close()
        input('\n 回车继续...\n')
    except Exception as ex:
        print('\t 访问数据错误：',ex)
        raise ex
#查找用户
def find(idkey):
    try:
        import sqlite3
        cn=sqlite3.connect('userinfo.db')
        cur=cn.execute('select * from user where uid=?',(idkey,))
        user=cur.fetchall()
        #存在返回 1,否则返回-1
        if len(user)>0:
            n=1
        else:
            n=-1
        cn.close()
```

```
        return n
    except Exception as ex:
        print('\t 数据库访问出错：',ex)
        raise ex
#重置用户数据
def resetdb():
    try:
        import sqlite3
        cn=sqlite3.connect('userinfo.db')
        cn.execute('drop table if exists user')
        cn.execute('create table user(uid text primary key,pwd text)')
        cn.close()
        print('\t 重置成功！')
        input('\n 回车继续...\n')
    except Exception as ex:
        print('\t 访问数据错误：',ex)
        raise ex
#显示菜单
while True:
    print('用户信息管理')
    print('\t1.显示用户')
    print('\t2.查找/修改/删除用户')
    print('\t3.添加用户')
    print('\t4.重置用户')
    print('\t5.退出系统')
    no=input('请选择: ')
    if no=='1':
        dispall()               #显示所有用户
    elif no=='2':
        seek_update()           #查找/修改/删除
    elif no=='3':
        adduser()               #添加用户
    elif no=='4':
        resetdb()               #重置用户
```

```
        elif no=='5':
            print('谢谢使用，再见!')
            break
    except Exception as ex:             #异常信息
        from traceback import print_tb
        from datetime import datetime
        log=open('Exp15_2.txt','a') #打开日志文件
        x=datetime.today()              #获取日期时间
        #显示异常信息
        print('\n 错误信息：')
        print('日期时间：',x)
        print('异常信息：',ex)
        print('堆栈跟踪信息：')
        print_tb(ex.__traceback__)
        #写入日志文件
        print('\n 错误信息：',file=log)
        print('日期时间：',x,file=log)
        print('异常信息：',ex.args[0],file=log)
        print('堆栈跟踪信息：',file=log)
        print_tb(ex.__traceback__,file=log)
        #关闭日志文件
        log.close()
        print('发生错误，退出!')
```

3．根据例 10.17，请自行完成。

例 10.17　利用 Java 和 SQL Server，实现 EBook 的 Emp（ENo,EName,ESex）中职工的输入、添加、修改、删除和查询等功能。参考代码如下：

```
import java.util.*;
import java.sql.*;
public class jdbcsql
{
    public static void main(String args[])
    {
        Connection cn=null;
        Statement st=null;
```

```
ResultSet rs=null;
String SqlStr;
int num=0;
Scanner in = new Scanner(System.in);
String vno=in.next();
String vname=in.next();
String vsex=in.next();
System.out.print(vno+"\t");
System.out.print(vname+"\t");
System.out.print(vsex+"\n");
try
{
    System.out.println("正在连接数据库...");
    Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
    cn=DriverManager.getConnection("jdbc:odbc:HSql", "sa", "");
    System.out.println("成功连接到数据库。");
    st=cn.createStatement();
    SqlStr="INSERT INTO Emp VALUES
        ('"+vno+"','"+vname+"','"+vsex+"')";
    num=st.executeUpdate(SqlStr);
    SqlStr="INSERT INTO Emp VALUES('E006','Tom','M')";
    num=st.executeUpdate(SqlStr);
    SqlStr="UPDATE Emp SET EName='Tim' WHERE EName='Tom'";
    num=st.executeUpdate(SqlStr);
    SqlStr="DELETE FROM Emp WHERE ENo='"+vno+"'";
    num=st.executeUpdate(SqlStr);
    rs=st.executeQuery("SELECT * FROM Emp");
    System.out.println("查询结果：");
    while (rs.next())
    {
        System.out.print(rs.getString("ENo") + "\t");
        System.out.print(rs.getString("EName") + "\t");
        System.out.print(rs.getString("ESex") + "\n");
```

```
            }
            rs.close();
            st.close();
            cn.close();
        } catch (Exception e)
            { System.err.println("异常："+e.getMessage()); }
    }
}
```